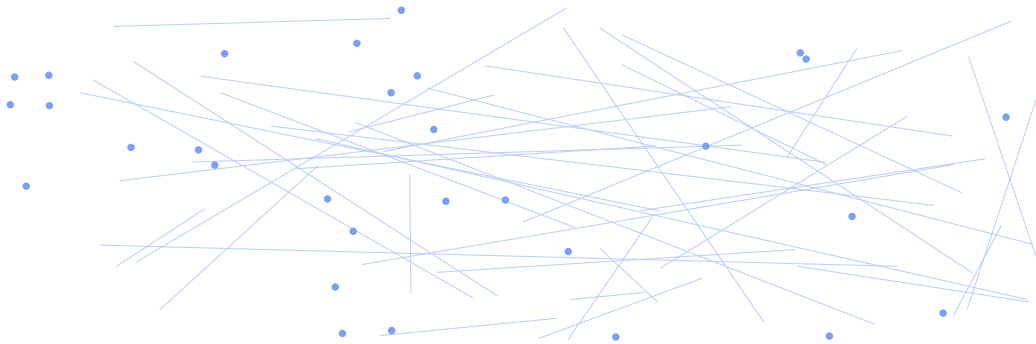




Computer Networks

Week 1 — Detailed Study Notes

Modules 1–3: Introduction, Latency & the Internet’s Shape, Links & Switching

IIT Madras BS — Course by Prof. Ayon Chakraborty

(Slides credited to Kurose & Ross, Prof. N.G. Srinivas, Prof. Sylvia Ratnasamy)

How to read these notes. Every slide concept appears here, in the order of the slides, grouped by module (Module 1 = Lecture 1, Module 2 = Lecture 2, Module 3 = Lecture 3). Coloured boxes mark the type of content:

- **Definition** — the precise term you must be able to state.
- **Intuition** — why the idea exists and what problem it solves.
- **Formula** — equations, with derivations and [unit] annotations.
- **Common pitfall** — mistakes students (and exam setters) love.
- **Beyond the slides** — standard textbook material (Kurose & Ross) needed to make the slide ideas quantitative; clearly flagged so you know what is extra.

The final part contains numerical problems with full solutions, subjective questions with model answers, and MCQs with explanations.

Contents

1	Module 1 — Introduction: What Is the Internet and Why Is It Hard?	3
1.1	The Internet is an exciting place	3
1.2	Evolution of Internet applications	3
1.3	Two “meanings” of the Internet	4
1.4	What is this course about? — Three pillars	4
1.5	What is a network?	5
1.5.1	Kinds of links	5
1.6	Challenge #1: How would multiple hosts communicate over a single link?	5
1.6.1	A single-link, multiple-access network	5
1.6.2	What must be solved even on a single link	6
1.7	Challenge #2: How would hosts across multiple links communicate?	7
1.8	Interconnecting many multi-link networks — and why it’s unreliable	8
1.8.1	The three failure modes, and where they come from	9
1.9	Challenge #3: How to guarantee application performance?	10
1.10	Challenge #4: How much data to push onto the link, at what rate?	10
1.11	Challenge #5: Resource crunch at the router	11
1.12	How the five challenges fit together	12
1.13	Components of a network: summary	12
1.14	Course logistics	12
1.14.1	Textbooks	12
1.14.2	Programming mini-projects (about three)	12
2	Module 2 — Latency, Throughput and the Shape of the Internet	13
2.1	Review of definitions	13
2.2	A toy experiment: How long should a message take from IIT Madras to IIT Delhi?	13
2.3	Reality check #1: <code>ping</code> shows 20–30× inflation	14
2.3.1	Decomposing the inflation	14
2.4	Reality check #2: <code>wget</code> — 754 ms for 200 KB	16
2.5	The bottleneck: Internet infrastructure	17
2.5.1	Internet $\neq$ Hyperloop	17
2.5.2	The hourglass: the Internet’s logical view	17
2.5.3	Protocol graph (the detailed version of the hourglass)	18
2.6	High-level goals for the coming weeks	19
3	Module 3 — How Do Machines Talk? Links, Topology and Switching	19
3.1	How do machines communicate? With 1s and 0s	19
3.2	Physical transmission on a single link	19

3.3	Topology: how do we scale a network to many end-systems?	21
3.3.1	(a) Full mesh: $O(n^2)$ links	21
3.3.2	(b) Single shared link: per-node capacity scales as $1/n$	22
3.3.3	(c) Switched networks enable efficient scaling	22
3.4	Critical nodes and critical links	22
3.5	Routers and multi-link networks: switching	23
3.6	Sharing a link: intuition with three bursty sources	24
3.6.1	Option 1 — Reservations: “Frequent overloading”	24
3.6.2	Option 2 — On demand: “No overloading”	25
3.6.3	Statistical multiplexing	25
3.6.4	What do we do under overload?	25
3.7	Two approaches to sharing: circuit switching vs packet switching	26
4	Formula Sheet	28
5	Numerical Problems with Worked Solutions	29
6	Subjective Questions with Model Answers	33
7	Multiple-Choice Questions with Explanations	35

1 Module 1 — Introduction: What Is the Internet and Why Is It Hard?

1.1 The Internet is an exciting place

The opening slide is a collage: Dropbox, Zoom, YONO SBI, Snapchat, MakeMyTrip, UPI (GPay, PhonePe, Paytm), Gmail, Instagram, Disney+ Hotstar, Bitcoin, AWS, Netflix, WhatsApp, Swiggy, Spotify — and, crucially, also an *industrial robot arm*, *remote robotic surgery*, a *power-grid substation*, and an *aircraft*.

Intuition: Why the collage matters

The point is not “apps are cool”. The point is that radically different applications — a UPI payment of Rs. 10, a 4K Netflix stream, a surgeon moving a robotic scalpel from another city, a grid controller tripping a breaker — *all ride on the same underlying infrastructure*. Their needs are wildly different:

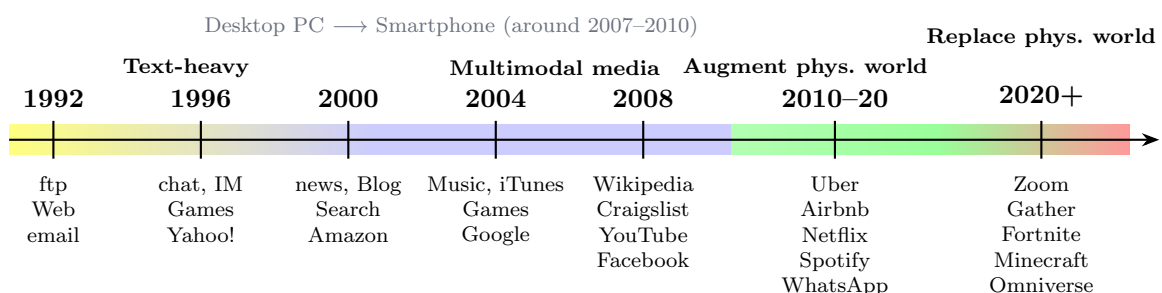
- UPI / banking: tiny data, must be **reliable** and **secure**; a few hundred ms is fine.
- Netflix / Hotstar: huge data, needs **throughput**; can buffer, so tolerates latency.
- Zoom / WhatsApp calls: moderate data, needs **low latency and low jitter**; tolerates some loss.
- Remote surgery / industrial robots / grid control: needs **bounded, predictable latency** and very high reliability — failures are physical and dangerous.

One network serving all of these is the central tension of the whole course.

1.2 Evolution of Internet applications

What the slide says (Lecture 1, slide 3)

“As Internet infrastructure and devices evolved, so did the applications.”

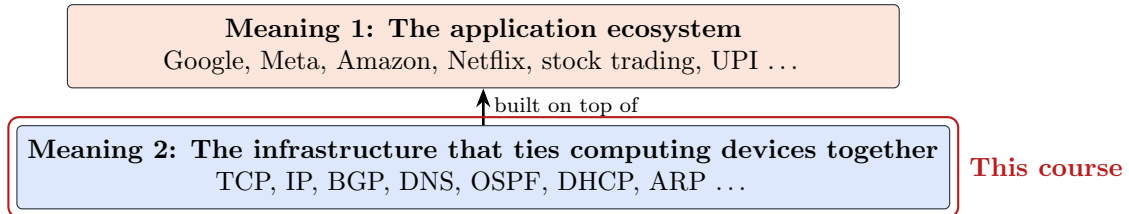


Intuition: Reading the timeline

- **Text-heavy era (1990s):** ftp, email, early Web pages. Kilobytes of data; delays of seconds were acceptable.
- **Multimodal media (2000s):** music, video, search, social networks. Bandwidth demand explodes.
- **Augmenting the physical world (2010–2020):** smartphones + GPS + always-on connectivity make Uber and Airbnb possible — the network now coordinates *physical* things (cars, rooms).
- **Replacing the physical world (2020+):** Zoom offices, Gather virtual spaces, Fortnite/Minecraft worlds, NVIDIA Omniverse digital twins. These need *interactive, low-latency, high-bandwidth* networking simultaneously.

The lesson: *applications follow infrastructure*. Each jump in link speed and device capability unlocked a new class of apps. Conversely, the next generation of apps (AR/VR, remote control, AI inference at the edge) will stress exactly the properties — latency and predictability — that the classic Internet was *not* designed to guarantee.

1.3 Two “meanings” of the Internet



Definition: The Internet (as used in this course)

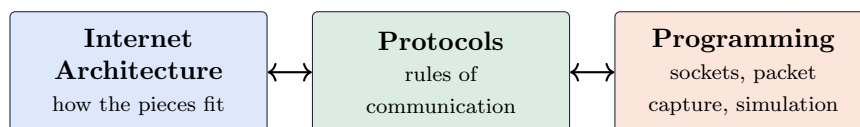
The **infrastructure** — links, routers, and the protocols (TCP, IP, BGP, DNS, OSPF, DHCP, ARP, ...) — that interconnects computing devices so that they can exchange data. *Not* the companies and services built on top.

Beyond the slides: What those acronyms are (preview)

You will meet all of these later; a one-line preview helps anchor them now:

Protocol	Full name	One-line job
TCP	Transmission Control Protocol	Reliable, ordered byte stream between two programs
IP	Internet Protocol	Addressing + best-effort delivery of packets host-to-host
BGP	Border Gateway Protocol	Routing <i>between</i> networks (e.g. Jio ↔ Airtel)
OSPF	Open Shortest Path First	Routing <i>within</i> one organisation's network
DNS	Domain Name System	Translate <code>iitm.ac.in</code> → an IP address
DHCP	Dynamic Host Configuration Protocol	Give a newly joined device an IP address
ARP	Address Resolution Protocol	Find the MAC address for an IP on the local link

1.4 What is this course about? — Three pillars



Intuition

Architecture is the *why*: design principles (layering, best effort, end-to-end argument). **Protocols** are the *what*: the concrete rules (message formats, state machines) that implement the architecture. **Programming** is the *how*: you will actually capture packets, write socket code in C, and build reliable delivery / MAC protocols. Understanding only one pillar gives you shallow knowledge; exams and projects test all three.

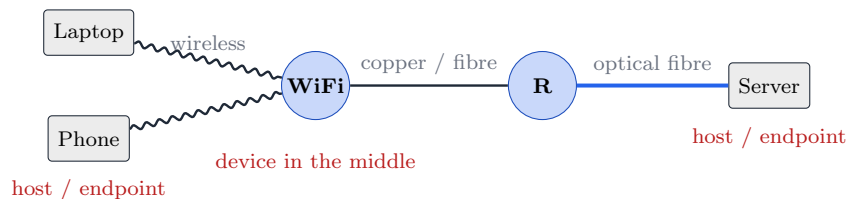
1.5 What is a network?

Definition: Network

A **carrier of information between two or more entities**. The entities are:

- **Hosts / endpoints** (the terms are used interchangeably): devices that run user applications — your laptop, phone, tablet, desktop.
- **Devices in the middle of the network**: e.g. your WiFi router, switches, core routers. They do not run your applications; they forward data.

The interconnection between entities is any physical medium capable of carrying information. Physical media are called **links**.



1.5.1 Kinds of links

Wireless links

Cellular 4G / 5G
 WiFi (IEEE 802.11)
 Bluetooth
 Satellite (e.g. Starlink, GEO satellites)

Wired links

Copper wire (twisted pair, e.g. Ethernet cable, telephone line)
 Lasers over optical fibre
 Coaxial cable (cable TV / DOCSIS internet)

Intuition: Why link type matters

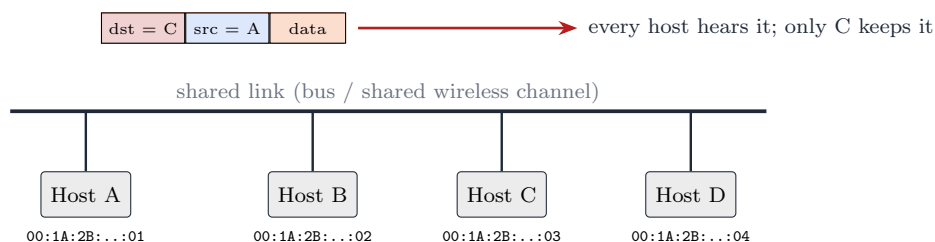
Every link has three physical properties you will use constantly:

1. **Bandwidth / rate** R (bits per second) — how fast bits can be pushed *onto* it.
2. **Propagation speed** s (metres per second) — how fast a signal travels *along* it ($\approx 3 \times 10^8$ m/s in air/vacuum, $\approx 2 \times 10^8$ m/s in fibre or copper).
3. **Error characteristics** — wireless links are noisy (interference, fading); fibre is extremely clean.

Wireless is also usually a *shared* (broadcast) medium: everyone in range hears everyone. That fact directly causes Challenge #1 below.

1.6 Challenge #1: How would multiple hosts communicate over a single link?

1.6.1 A single-link, multiple-access network



What the slide says (Lecture 1, slide 8)

- Send bits of data in **packets** or frames.
- How do we differentiate among many receivers?
- Every endpoint has a link-level **address**: also called a *MAC* address.
- Packets have a destination address on them.
- However, we can't have every computer in the world on the same link! — physical limits on power / distance over which information travels over a single link.

Definition: Packet / frame

A **packet** is a bounded unit of data transmission (e.g. up to ≈ 1500 bytes on Ethernet). At the link level it is usually called a **frame**. It carries a *header* (addresses, control info) and a *payload* (the data).

Definition: MAC address

A **link-level address** (Medium Access Control address) that identifies an interface on a link. Ethernet/WiFi MAC addresses are 48 bits, written as six hex bytes, e.g. 3C:22:FB:9A:10:7E.

Intuition: Why packets, and why addresses?

Why chop data into packets? (1) Fairness: a 2 GB file sent as one blob would monopolise the shared link; small packets let many hosts interleave. (2) Error recovery: if one bit is corrupted, you resend 1500 bytes, not 2 GB. (3) Buffering: devices in the middle only need memory for a few packets, not whole files.

Why addresses? On a shared medium, *every* host physically receives *every* transmission — think of a classroom where everyone hears whatever is said. The only way to direct a message is to prefix it with a name (“Ravi, ...”). The destination MAC address is exactly that name; each host's network card compares it with its own address and silently discards frames not meant for it.

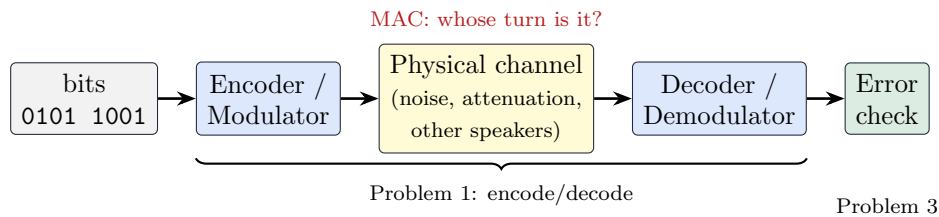
Intuition: Why not one giant link for the whole world?

Signals weaken (attenuate) with distance and must be above noise to be decoded; so a single link has a maximum physical reach. Also, a shared link's capacity is divided among everyone on it: with n hosts sharing rate R , each gets on average R/n (see Module 3). With billions of hosts, each would get essentially nothing, and collisions would be constant. So we must build *networks of links* — which is Challenge #2.

1.6.2 What must be solved even on a single link**What the slide says (Lecture 1, slide 9)**

Even on a single link, you need to worry about a few things:

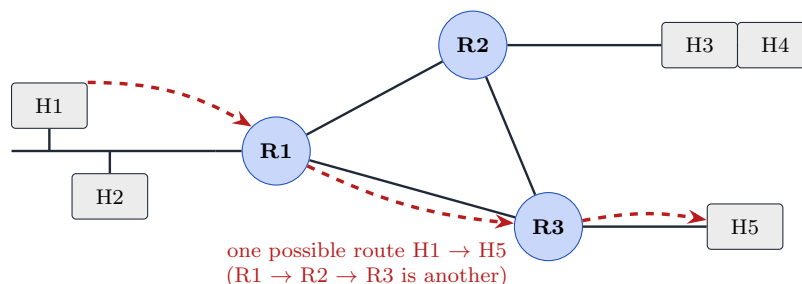
1. Converting digital data to physical signals over the medium (**encode/decode**).
2. How do we decide who speaks? (**medium access control problem**).
3. **Detecting and correcting errors**.



Intuition: The three single-link problems

1. **Encode/decode.** A computer holds bits; a wire carries voltages, a fibre carries light, the air carries radio waves. Some agreed mapping between bits and physical signals is needed (Module 3 covers amplitude, frequency and phase modulation).
2. **Medium access control (MAC).** If two hosts talk at the same time on a shared medium, their signals add up and *collide* — neither is decodable. We need a rule for *who speaks when*: take turns (like a token), divide the channel (time/frequency slots), or “listen before talking and back off on collision” (like Ethernet’s CSMA/CD or WiFi’s CSMA/CA). This is exactly human conversation etiquette, formalised.
3. **Error detection/correction.** Noise flips bits. Add redundant bits (checksums, CRCs) so the receiver can *detect* corruption, or error-correcting codes so it can *fix* some errors without retransmission.

1.7 Challenge #2: How would hosts across multiple links communicate?



What the slide says (Lecture 1, slide 11)

- Connect multiple links via **routers**.
- Need to figure out how to move packets from one host to another host, e.g. how to reach google.com from your laptop.
- Known as the **routing problem**.
- **Key Q:** How should packets be moved from A to reach B?

Definition: Router

A machine that processes packets, moving them from one link (input) to another link (output) towards their destination.

Definition: Routing problem

Determining, for every destination, the path (sequence of routers and links) a packet should take from its source.

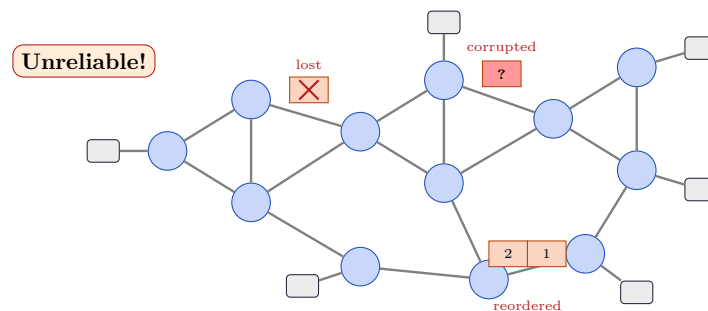
Intuition: Routing = navigation for packets

Think of India's road network. A truck from Chennai to Delhi doesn't carry a full map; at each junction a signboard says "Delhi → take NH44". Routers are the junctions; their *forwarding tables* are the signboards. The routing problem is the question of *who writes the signboards and how*, especially when roads (links) close or get jammed.

It is harder than GPS navigation because: (1) nobody has a global map of the Internet (tens of thousands of independently owned networks), (2) the map changes constantly (links fail, new routes appear), (3) decisions must be made in nanoseconds per packet, and (4) owners have *policies* ("never route my customers' traffic through my competitor").

Beyond the slides: Routing vs forwarding — a distinction you will need soon

Forwarding is the *local, per-packet, fast* action: look up the destination in a table and push the packet out of the correct port (nanoseconds, in hardware). **Routing** is the *global, slow, network-wide* process of computing those tables (seconds to minutes, in software, using protocols like OSPF and BGP). Analogy: forwarding = turning at one junction; routing = planning the whole trip.

1.8 Interconnecting many multi-link networks — and why it's unreliable**What the slide says (Lecture 1, slide 13)**

In general, networks give no guarantees.

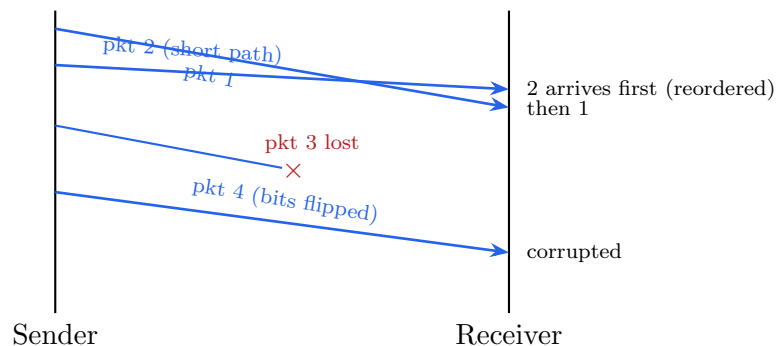
- Packets may be *lost*, *corrupted*, *reordered* on the way to the destination.
- **Best effort delivery.**
- Advantage: the network becomes very simple to build — don't have to make it reliable; don't need to implement any performance guarantees; don't need to maintain packet ordering; almost any medium can deliver individual packets.
- Early Internet thrived: easy to engineer, no guarantees to worry about.

Definition: Best-effort delivery

The network *tries* to deliver each packet but makes **no promise** about whether it arrives, when it arrives, whether its contents are intact, or in what order packets arrive relative to each other.

1.8.1 The three failure modes, and where they come from

Failure	Physical cause	Example
Loss	Router buffer full $\Rightarrow$ packet dropped; link failure; bit errors so bad the frame is discarded	Your video call freezes for a moment
Corruption	Noise / interference flips bits in transit	WiFi near a microwave oven
Reordering	Packets of one flow take different paths with different delays, or are re- sent	Packet 2 arrives before packet 1
(Duplication)	A retransmission when the original was only delayed, not lost	Same packet delivered twice



Intuition: Why “no guarantees” was a brilliant design choice

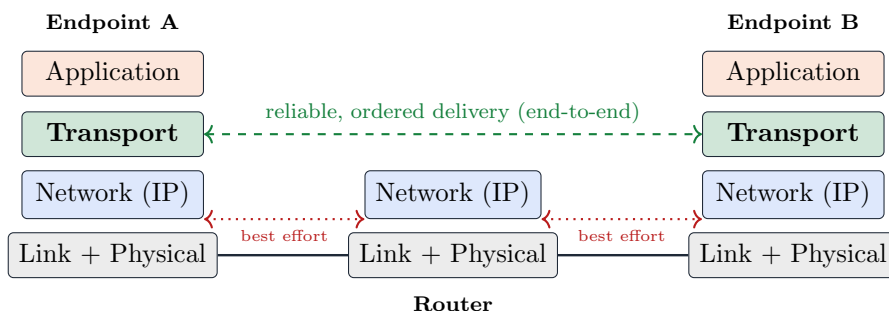
Compare the Indian Post’s ordinary letters with a registered-post-with-acknowledgement service. Ordinary post is cheap and simple precisely because it does not track, confirm or order anything. The Internet’s designers made the core network like ordinary post and pushed any extra guarantees to the *edges* (the end hosts), which only need them sometimes.

Consequences:

- **Simplicity of the core:** routers keep no per-connection state; they just forward. Cheap, fast, scalable.
- **Universality:** “almost any medium can deliver individual packets” — radio, fibre, copper, satellite, even (famously, as an RFC joke) carrier pigeons. The Internet could glue together wildly different networks.
- **Robustness:** if a router dies, packets simply take another path; no “connection” was lost in the core.
- **Innovation:** new applications don’t need permission or new network features — they just need packets.

The price: applications that *need* guarantees must build them on top. That’s Challenge #3.

1.9 Challenge #3: How to guarantee application performance?



What the slide says (Lecture 1, slide 15)

- How should endpoints provide guarantees to applications?
- **Transport** software on the endpoint oversees implementing guarantees on top of an unreliable network.
- Need to solve the **reliable data delivery** problem.
- For some applications, also need **ordered delivery**.

Intuition: Building reliability out of unreliability

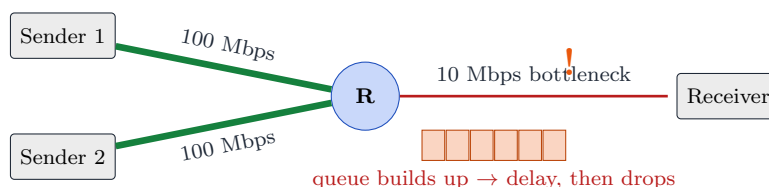
Note in the diagram: the router has *no transport layer*. Guarantees are implemented only in the two endpoints — this is the famous **end-to-end principle**. How can two endpoints create reliability over a lossy network? The same way you'd send an important document by ordinary post:

- **Number** every packet (sequence numbers) $\Rightarrow$ the receiver can detect gaps and reorder.
- Receiver sends **acknowledgements (ACKs)** $\Rightarrow$ the sender knows what arrived.
- Sender uses a **timer**; no ACK in time $\Rightarrow$ **retransmit**.
- Add a **checksum** $\Rightarrow$ receiver detects corruption and treats it as a loss.

This is exactly what TCP does (you'll build a version in Programming Project 3).

Reliable $\neq$ ordered. Reliability means every byte eventually arrives; ordering means bytes are handed to the application in the sequence they were sent. A file download needs both. A live video call often wants *neither* strictly: a late frame is useless, so it's better to skip it (hence UDP-based protocols for real-time media).

1.10 Challenge #4: How much data to push onto the link, at what rate?



What the slide says (Lecture 1, slide 17)

- How quickly should endpoints send data?
- Known as the **congestion control** problem.
- Congestion control algorithms at source endpoints react to remote network congestion. Part of the transport sw/hw stack.
- Key question: How to vary the sending rate based on network signals?

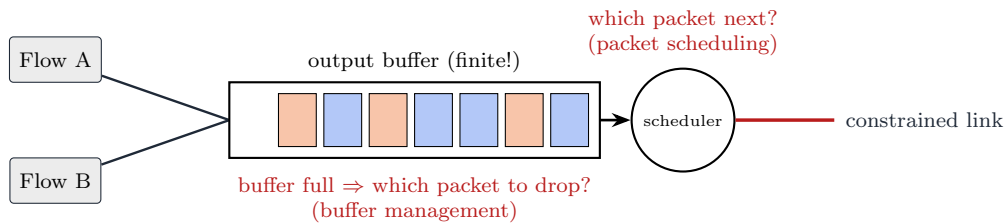
Intuition: Traffic on a highway

The slide's photo is a jammed highway. If every car enters the highway as fast as it can, the merge point jams and *everyone* slows down — total useful throughput can even *collapse*. Each sender only sees its own access link (100 Mbps) and has no idea that somewhere far away there is a 10 Mbps bottleneck shared with others.

The network (best-effort!) will not tell the sender its fair rate. So the sender must *infer* congestion from **signals**: packet losses (buffer overflowed), rising delay (queues growing), or explicit marks (ECN). Then it **adjusts its rate**: slow down on congestion, probe upward when things look fine. TCP's famous “additive increase, multiplicative decrease” (AIMD) is one such algorithm; later weeks cover it.

Flow control vs congestion control (don't confuse them): flow control protects the *receiver* from being overrun; congestion control protects the *network*.

1.11 Challenge #5: Resource crunch at the router



What the slide says (Lecture 1, slide 19)

- How should a router transmit packets when network resources are scarce?
- Known as the **packet scheduling** problem.
- **Key question**: which packet to transmit over a constrained network link, and when?
- Related: the **buffer management** problem.

Intuition: The router as a bank queue

Challenge #4 was the *sender's* view of congestion; Challenge #5 is the *router's* view. Packets arrive faster than the outgoing link can transmit them, so they wait in a buffer. Two decisions:

- **Scheduling**: in what order do waiting packets leave? First-In-First-Out (FIFO) is simplest. But a video-call packet stuck behind 50 bulk-download packets suffers. Alternatives: *priority queues* (urgent traffic first), *round robin* / *fair queuing* (each flow gets a turn — like separate counters in a bank).
- **Buffer management**: when the buffer is full, who gets dropped? The arriving packet (“drop-tail”)? A random one early, before the buffer is full, as a warning signal to senders (RED)? A packet from the flow hogging the most space?

Bigger buffers aren't a free fix: they reduce drops but increase *queuing delay* — the “bufferbloat” problem that makes gaming and video calls laggy on home routers.

1.12 How the five challenges fit together

Applications — Web, email, streaming, payments ...

Transport (endpoints) — Challenge #3: reliable & ordered delivery;
Challenge #4: congestion control

Inside routers — Challenge #5: packet scheduling & buffer management

Network — Challenge #2: routing across multi-link networks (best effort)

Link / Physical — Challenge #1: encode/decode,
medium access control, error detection (single link)

What the slide says (Lecture 1, slide 20)

“Throughout this course we will study the design principles adopted to address such challenges at the Internet scale.”

1.13 Components of a network: summary

Component	Definition (from slides)
Link	Communication link for transmission — a physical medium (wired or wireless).
Host / Endpoint	Computer running applications of the end user.
Router	Computer for routing packets from an input link to another output link.
Network	A group of hosts, links, routers capable of sending packets among its members.

1.14 Course logistics

1.14.1 Textbooks

- **Primary:** *Computer Networking: A Top-Down Approach*, 7th ed., Kurose & Ross (**KR**).
- *Computer Networks*, Andrew Tanenbaum (**AT**).
- Network programming: *UNIX Network Programming, Vol. 1, 3rd ed.: The Sockets Networking API*, Richard Stevens (**RS**).
- **Slides will NOT be enough for a good understanding.** (These notes fill many gaps, but for Week 1 read KR Chapter 1, sections 1.1–1.4.)

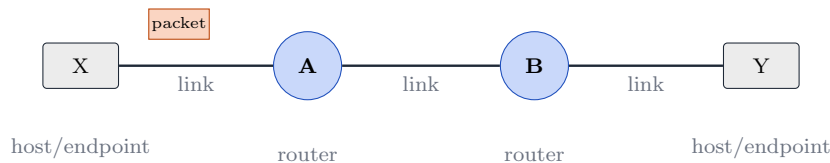
1.14.2 Programming mini-projects (about three)

Project	Language	Topic
P1	Python	Capturing and analysing network packets (think Wireshark / <code>scapy</code> / raw sockets)
P2	C	Socket programming (the Stevens book is your friend)
P3	Python	Reliable data delivery <i>or</i> MAC protocols

- Encouraged to work in **groups of two** — to appreciate your code running on multiple systems across a network.
- A **Linux laptop is required**. TAs/instructor will *not* assist with WSL-specific problems.

Common pitfall: Practical note on the Linux requirement

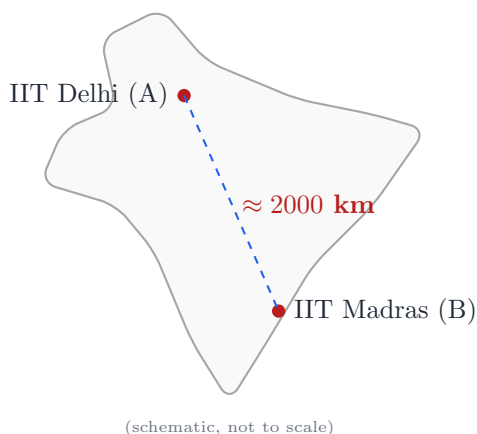
WSL2 runs behind a virtual NAT, so raw packet capture and two-machine socket experiments behave differently from native Linux (extra hop, different interfaces, firewall on the Windows side). If you only have Windows, a dual-boot or a native Linux install on a spare machine will save you debugging time on P1 and P2.

2 Module 2 — Latency, Throughput and the Shape of the Internet**2.1 Review of definitions**

Term	Definition
Endpoint / Host	Machine running a user application.
Packet	A unit of data transmission (e.g. 1500 bytes).
Link	A physical communication channel between two or more machines.
Router	A machine that processes packets, moving them from one link to another towards a destination.
Network	Collection of interconnected machines.
Address	A unique name given to a machine (more later).

Intuition: Why 1500 bytes?

1500 bytes is the standard Ethernet *MTU* (Maximum Transmission Unit) — the largest payload an Ethernet frame carries. It's a historical compromise: big enough that header overhead is small (≈ 40 bytes of TCP/IP headers is under 3%), small enough that one packet doesn't hog a shared link for long and a single bit error doesn't waste much.

2.2 A toy experiment: How long should a message take from IIT Madras to IIT Delhi?**Ideal latency (speed of light)**

$$t = \frac{\text{distance}}{\text{speed}} = \frac{2000 \text{ km}}{c}$$

$$= \frac{2 \times 10^6 \text{ m}}{3 \times 10^8 \text{ m/s}} \approx 6.67 \text{ ms}$$

```
$ ping iitd.ac.in
```

```
$ wget iitd.ac.in
```

Formula: Propagation delay (the “speed of light” latency)

$$d_{\text{prop}} = \frac{D}{s} \quad \left[s = \frac{\text{m}}{\text{m/s}} \right]$$

where D = distance travelled by the signal, s = propagation speed in the medium. For the slide: $D = 2000 \text{ km} = 2 \times 10^6 \text{ m}$, $s = c = 3 \times 10^8 \text{ m/s}$:

$$d_{\text{prop}} = \frac{2 \times 10^6}{3 \times 10^8} \text{ s} = 6.67 \times 10^{-3} \text{ s} \approx 6.6 \text{ ms.}$$

Definition: Latency

The time taken for (the first bit of) a message to get from source to destination. The slide calls the physics lower bound D/c the latency; in reality latency includes several components (below).

2.3 Reality check #1: ping shows 20–30× inflation**What the slide says (Lecture 2, slide 7)**

Running `ping iitd.ac.in` from IIT Madras shows “**20–30× inflation with respect to c-speed!**”

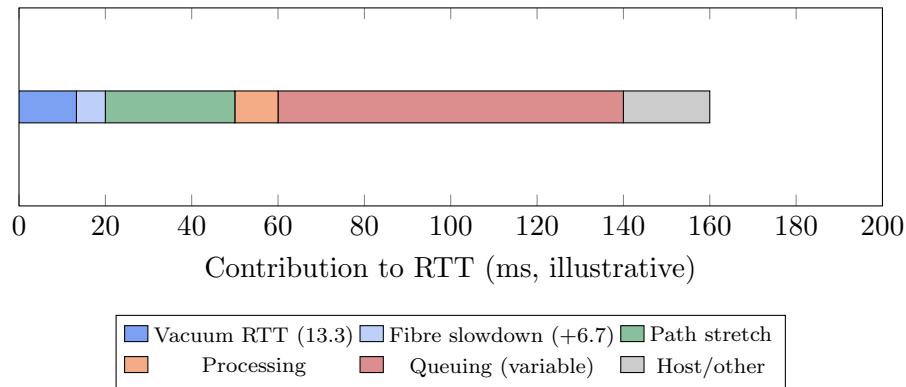
So the measured value is roughly $20 \times 6.6 \approx 130 \text{ ms}$ to $30 \times 6.6 \approx 200 \text{ ms}$. Where does a factor of 20–30 come from? Let’s decompose it honestly.

Common pitfall: An apples-to-oranges comparison hidden in the slide

ping reports the **round-trip time (RTT)**: request goes $B \rightarrow A$ and the reply comes $A \rightarrow B$. The 6.6 ms figure is a **one-way** time. So a factor of **2** of the “inflation” is just accounting. Always check whether a number is one-way or round-trip!

2.3.1 Decomposing the inflation

1. **Round trip vs one way:** $\times 2$. Fair baseline RTT in vacuum = $2 \times 6.67 = 13.3 \text{ ms}$.
2. **Light is slower in fibre:** $\times 1.5$. Glass has refractive index $n \approx 1.47$, so light travels at $c/n \approx 2 \times 10^8 \text{ m/s}$. Fibre RTT baseline = $2 \times \frac{2 \times 10^6}{2 \times 10^8} = 20 \text{ ms}$.
3. **Cables don’t follow straight lines (path stretch):** $\times 1.5$ – 3 . Fibre is laid along railways and highways, through cities, and traffic often goes through exchange points in other cities (e.g. Chennai $\rightarrow$ Mumbai $\rightarrow$ Delhi), or in bad cases even *out of the country and back* if two networks don’t peer in India.
4. **Per-hop processing delay.** Each router examines the header, looks up the forwarding table, checks for errors. Usually microseconds, but there may be 10–20 hops.
5. **Queuing delay.** Packets wait in router buffers behind other traffic (Challenge #5). This is highly variable — which is why successive pings show different times.
6. **Transmission delay.** Time to push the packet’s bits onto each link (tiny for a 64-byte ping on fast links).
7. **End-host overheads.** The ICMP reply is generated by the remote host’s OS; low priority, some ms.



The split above is illustrative (actual values depend on the path on the day of measurement); the first two bars are exact physics.

Beyond the slides: The four components of nodal delay (KR §1.4)

The slide's "Time = Distance/Speed" is only one of four delays a packet experiences at each hop:

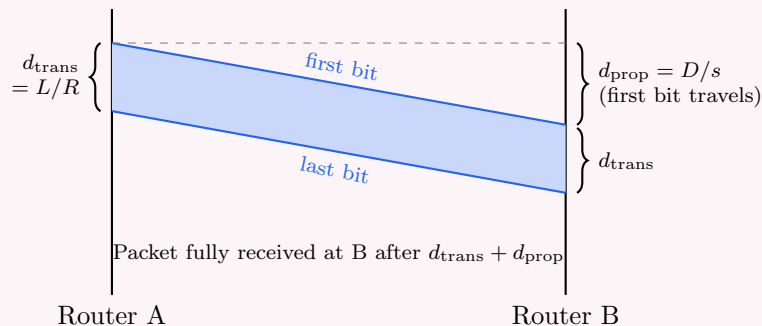
Formula: Total nodal delay at one hop

$$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}} \quad [\text{s}]$$

- d_{proc} : processing (header check, lookup) — typically μs .
- d_{queue} : waiting in the output buffer — 0 to many ms, depends on load.
- $d_{\text{trans}} = \frac{L}{R}$ $\left[\text{s} = \frac{\text{bits}}{\text{bits/s}} \right]$ where L = packet length, R = link rate.
- $d_{\text{prop}} = \frac{D}{s}$ $\left[\text{s} = \frac{\text{m}}{\text{m/s}} \right]$.

Derivation / intuition for L/R . A link of rate R bits/s places one bit on the wire every $1/R$ seconds. To put all L bits on the wire takes $L \times (1/R) = L/R$ seconds.

Transmission vs propagation — the toll booth analogy (KR). A caravan of 10 cars (bits of one packet) goes through a toll booth that serves one car every 12 s (link rate). The time for the whole caravan to pass the booth is $10 \times 12 = 120$ s — that's transmission delay; it depends on the *packet size and rate*, not on distance. The time for a car to drive 100 km to the next booth at 100 km/h is 1 hour — that's propagation delay; it depends on *distance and speed*, not on packet size. Students confuse these all the time.



2.4 Reality check #2: wget — 754 ms for 200 KB

What the slide says (Lecture 2, slide 9)

Running `wget iitd.ac.in` (download the homepage): “**754 ms for 200 KBs! Avg. Throughput: 265 KB/sec**”.

Definition: Throughput

The rate at which data is actually delivered to the receiver, measured in bits (or bytes) per second:

$$\text{Throughput} = \frac{\text{amount of data transferred}}{\text{time taken}} \quad \left[\frac{\text{bytes}}{\text{s}} \text{ or } \frac{\text{bits}}{\text{s}} \right]$$

Formula: Verifying the slide’s number

$$\text{Throughput} = \frac{200 \text{ KB}}{0.754 \text{ s}} = 265.25 \text{ KB/s} = 265.25 \times 8 \text{ kbit/s} \approx 2.12 \text{ Mbps.}$$

Units: B = byte, b = bit; 1 B = 8 b. Networking rates are almost always quoted in *bits/s* (Mbps); file sizes in *bytes*. Converting is the #1 source of exam errors.

Intuition: Why only 2 Mbps when the campus links are gigabit?

Latency and throughput are **different** metrics — latency is “how long until the first bit arrives”, throughput is “how many bits per second once flowing”. But for a *short* transfer, latency dominates throughput:

- DNS lookup for `iitd.ac.in` (maybe 1 RTT, unless cached).
- TCP connection setup (3-way handshake): 1 RTT before any data.
- HTTP request → first byte of response: 1 RTT.
- **TCP slow start:** TCP doesn’t know the network’s capacity, so it starts sending a small window and doubles it every RTT. For 200 KB, it never reaches full speed — the transfer is over first.

Beyond the slides: A back-of-envelope model that reproduces 754 ms

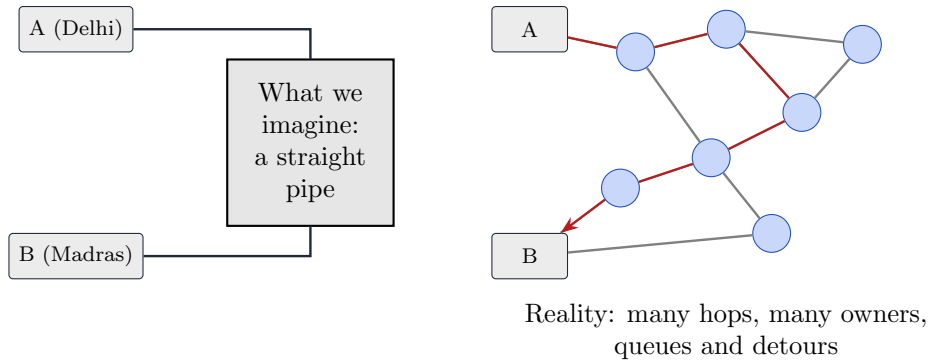
Assume $\text{RTT} \approx 150 \text{ ms}$ (consistent with the ping result), max segment size (MSS) = 1460 bytes, TCP initial window of 10 segments $\approx 14.6 \text{ KB}$, doubling each RTT:

RTT #	Activity	Data sent this RTT	Cumulative
1	TCP handshake (SYN, SYN-ACK)	0	0
2	HTTP GET + first window	14.6 KB	14.6 KB
3	window doubled	29.2 KB	43.8 KB
4	doubled	58.4 KB	102.2 KB
5	doubled	116.8 KB	219 KB $\geq 200 \text{ KB}$ ✓

Total $\approx 5 \times 150 = 750 \text{ ms}$ — almost exactly the 754 ms on the slide. The “throughput” of 265 KB/s is therefore *not* a property of the links; it’s an artefact of $\text{RTT} \times \text{number of round trips}$. This is a preview of TCP (later weeks) — the takeaway for now: **for small transfers, latency (RTT) is the bottleneck, not bandwidth.**

2.5 The bottleneck: Internet infrastructure

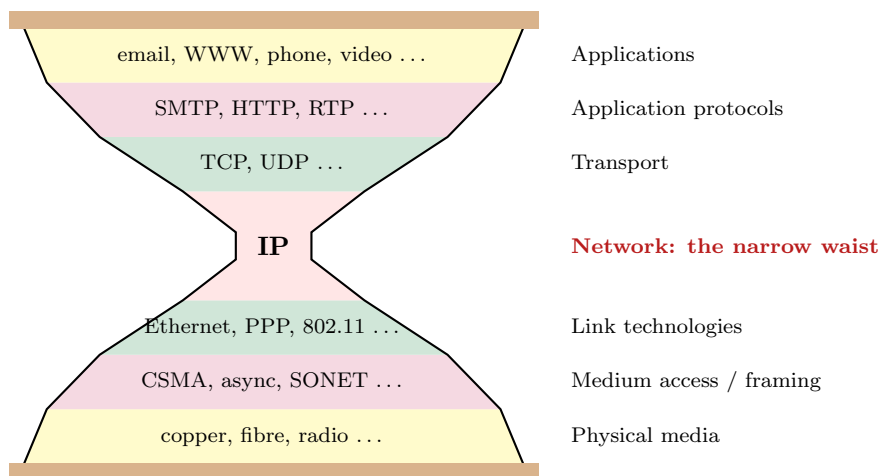
2.5.1 Internet $\neq$ Hyperloop



Intuition

The slide's joke: the Internet is not a vacuum tube carrying data at light speed from Delhi to Madras. It is a patchwork of links and routers owned by different organisations (NKN/ERNET for academic networks, Jio, Airtel, BSNL, Tata Communications, international carriers), glued together by protocols. The inflation we measured comes from this structure.

2.5.2 The hourglass: the Internet's logical view



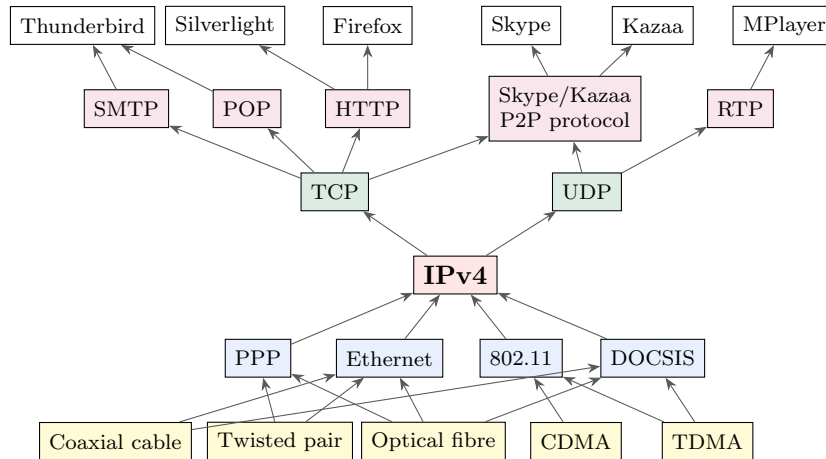
Intuition: Why an hourglass? — the “narrow waist”

At the top there are thousands of applications; at the bottom, dozens of physical technologies. In the middle, exactly **one** protocol: **IP**. This is the most important architectural idea in the course:

- **Without a waist**, every application would need to be written for every link technology: $M \text{ apps} \times N \text{ technologies} = MN \text{ adaptations}$.
- **With a single common layer**, apps target IP and each link technology implements IP once: $M + N \text{ adaptations}$. (Same reason a universal power adapter or a common language is useful.)
- **Innovation above and below is decoupled**. WiFi 7 or 5G can be deployed without changing any app; a new app (say, a new video codec protocol) works over every existing link.
- **The cost**: IP is hard to change (IPv4 $\rightarrow$ IPv6 has taken 25+ years), and IP offers only

best effort — so whatever performance limitations IP has, *all* apps inherit. That's why the slide calls the Internet infrastructure “the bottleneck”.

2.5.3 Protocol graph (the detailed version of the hourglass)



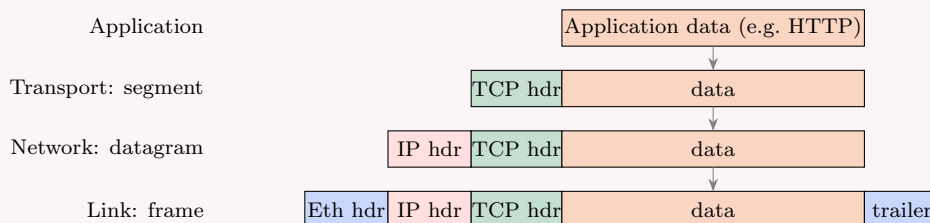
Intuition: Reading the protocol graph

Arrows point upward: each protocol is *used by* the ones above it. Notice that *every* path from the bottom to the top passes through the single IPv4 box — that is the hourglass waist, now drawn as a graph. Also notice:

- HTTP, SMTP, POP ride on **TCP** (they need reliable, ordered bytes: web pages, emails).
- RTP (real-time media, e.g. MPlayer streaming) rides on **UDP** (timeliness over reliability).
- Skype/Kazaa P2P use **both**.
- Ethernet can run over coax, twisted pair or fibre — the link layer itself is independent of the physical medium.

Beyond the slides: Encapsulation — how layering looks on the wire

Each layer wraps the data from the layer above in its own header, like envelopes inside envelopes:



Routers only open the envelopes up to the IP header (network layer); endpoints open all of them.

2.6 High-level goals for the coming weeks

What the slide says (Lecture 2, final slide)

- Dive a bit deeper into how Internet communication works:
 - **Links:** how does communication work physically?
 - **Routers:** how do they move data between links?
 - **Endpoints:** how is networking organised at endpoints?
- Understand how to **measure** the Internet.
- Guiding questions: *What is a network made of? How is it shared? How do we evaluate a network?*

Summary: How we evaluate a network — the core metrics

Metric	Meaning	Unit	Measured by
Latency (one-way)	time for a bit to go A→B	s, ms	timestamps (needs synchronised clocks)
RTT	A→B→A	s, ms	ping
Throughput	delivered data rate	bit/s	wget, iperf
Loss rate	fraction of packets lost	%	ping statistics
Jitter	variation in delay	ms	successive pings
Path / hops	routers on the way	count	traceroute

Try these yourself: `ping -c 10 iitd.ac.in`, `traceroute iitd.ac.in`, `wget iitd.ac.in` — it's the spirit of the “measure the Internet” goal and good practice for P1.

3 Module 3 — How Do Machines Talk? Links, Topology and Switching

3.1 How do machines communicate? With 1s and 0s

What the slide says (Lecture 3, slide 2)

- With 1s and 0s — computers only deal with 1s and 0s.
- How do we transmit 1s and 0s in a network? $\Rightarrow$ the **encoding and decoding problem**.

3.2 Physical transmission on a single link

What the slide says (Lecture 3, slide 3)

- Physical signalling (light, AC voltages, etc.) is often **analog**.
- Convert bits to signals through **modulation** of the physical characteristics of signals: **encoding**.
- Convert signals back to digital by **decoding** physical signals.

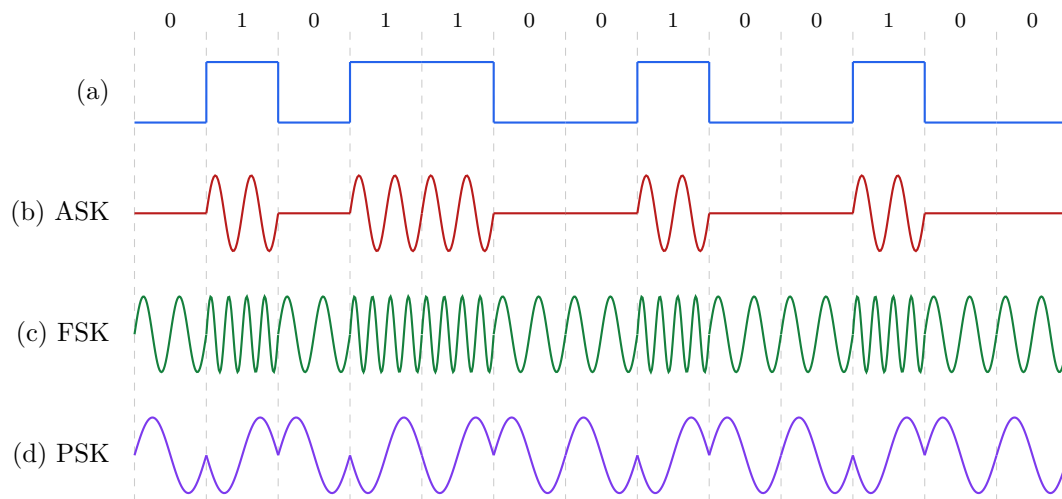
Figure (Tanenbaum Fig. 2-18): (a) a binary signal, (b) amplitude modulation, (c) frequency modulation, (d) phase modulation.

Definition: Carrier wave

A sinusoid $s(t) = A \sin(2\pi ft + \phi)$ that is used to carry information. It has exactly three knobs: **amplitude** A [V], **frequency** f [Hz = s⁻¹], **phase** ϕ [rad].

Definition: Modulation

Varying one (or more) of the carrier's parameters according to the data bits. Encoding = modulation at the sender; decoding = demodulation at the receiver (detecting which parameter value was sent in each bit period).



	Amplitude (ASK)	Frequency (FSK)	Phase (PSK)
What varies	A : e.g. A_1 for 1, 0 (or A_0) for 0	f : f_1 for 1, f_0 for 0	ϕ : 0 for 0, π for 1
Signal	$A_b \sin(2\pi f_c t)$	$A \sin(2\pi f_b t)$	$A \sin(2\pi f_c t + \phi_b)$
Noise robustness	Worst — noise directly changes amplitude	Better — noise rarely shifts frequency	Best (for binary) — phases are maximally separated
Where used	Optical fibre on/off keying (light on = 1)	Old modems, Bluetooth (GFSK), pagers	WiFi, 4G/5G (as part of QAM), satellite

Intuition: Why modulate at all? Why not just send voltages 0 and 5?

For a short copper wire you can (the “binary signal” in (a), called baseband signalling). But:

- Light in fibre and radio waves in the air *are* oscillations — there is no “DC” light or “DC” radio. You must use a carrier and vary its properties.
- Different users can use different carrier frequencies simultaneously without interfering (FM radio stations, WiFi channels) — this is *frequency-division* sharing.
- Antennas must be comparable in size to the wavelength; high-frequency carriers allow small antennas.

Decoding means the receiver, in each bit period, decides: “was the amplitude high or low?” (ASK), “was it the fast or the slow wave?” (FSK), “was the wave inverted?” (PSK). Noise makes these decisions uncertain $\Rightarrow$ bit errors $\Rightarrow$ the need for error detection (Challenge #1, part 3).

Beyond the slides: Symbols, baud and bit rate

One “bit period” in the figure is one **symbol**. Binary schemes carry 1 bit per symbol. If a scheme uses M distinguishable signal states (e.g. 4 phases in QPSK), each symbol carries $\log_2 M$ bits:

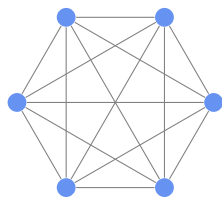
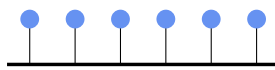
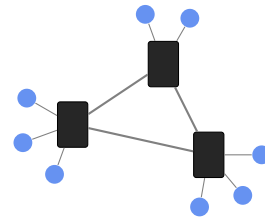
Formula: Bit rate vs symbol (baud) rate

$$R_{\text{bit}} = R_{\text{symbol}} \cdot \log_2 M \quad \left[\frac{\text{bits}}{\text{s}} = \frac{\text{symbols}}{\text{s}} \cdot \frac{\text{bits}}{\text{symbol}} \right]$$

Derivation: M distinguishable states can label M different bit patterns; k bits have 2^k patterns, so $2^k = M \Rightarrow k = \log_2 M$ bits per symbol. E.g. 1000 symbols/s with 16 states (16-QAM) gives $1000 \times 4 = 4000$ bit/s. Modern WiFi uses 1024-QAM ($\log_2 1024 = 10$ bits/symbol) on clean channels. The catch: more states packed into the same power budget are closer together, so noise causes more errors — rate and robustness trade off.

3.3 Topology: how do we scale a network to many end-systems?

Two machines need one link. What about n machines? The slides compare three designs.

**(a) Full mesh** $O(n^2)$ links**(b) Single shared link**per-node capacity $\propto 1/n$ **(c) Switched network**

efficient scaling

3.3.1 (a) Full mesh: $O(n^2)$ links**Formula: Number of links in a full mesh**

Each of the n nodes connects to the other $n - 1$. Counting from every node gives $n(n - 1)$, but every link has been counted twice (once from each end), so

$$\# \text{links} = \binom{n}{2} = \frac{n(n - 1)}{2} = O(n^2) \quad [\text{links}]$$

Each node needs $n - 1$ network interfaces (ports).

Numbers: $n = 10 \Rightarrow 45$ links; $n = 1000 \Rightarrow 499,500$ links; $n = 10^9$ (Internet-scale) $\Rightarrow \approx 5 \times 10^{17}$ links. Obviously impossible.

Intuition

The full mesh gives every pair a dedicated link: best possible performance, no sharing, no routing, highest fault tolerance. But cost grows quadratically, and adding the $(n + 1)$ -th node requires n new cables *and* a new port on every existing node. It's like wanting a private road between every pair of houses in a city.

3.3.2 (b) Single shared link: per-node capacity scales as $1/n$

Formula: Per-node capacity on a shared link

If n nodes share a link of capacity R and all want to send, the best they can each get on average is

$$R_{\text{per node}} = \frac{R}{n} \quad \left[\frac{\text{bits/s}}{\text{nodes}} \right]$$

(and in practice less, because of collisions and MAC overhead).

Intuition

Cheap (one cable, one port per node) but it doesn't scale: 1 Gbps shared by 1000 nodes is 1 Mbps each at best; plus the medium access control problem gets harder with more speakers, and physical reach is limited (Module 1).

3.3.3 (c) Switched networks enable efficient scaling

Intuition

The switched network is the middle ground: a modest number of links, with **switches/routers** (black boxes) forwarding data between them. Each host needs just *one* link to its nearest switch; switches are interconnected by high-capacity links. Links are shared — but only by the traffic that actually needs them, not by everyone. This is how the telephone network and the Internet are both built. The price: switches must now *route* and *switch* (Challenge #2 and the rest of this module).

	Full mesh	Shared link	Switched
Links for n hosts	$n(n-1)/2$	1	$\approx n+$ (few switch links)
Ports per host	$n-1$	1	1
Capacity per host	full R to each peer	R/n	depends on load and path
Needs routing?	No	No (needs MAC)	Yes
Scales?	No	No	Yes

3.4 Critical nodes and critical links

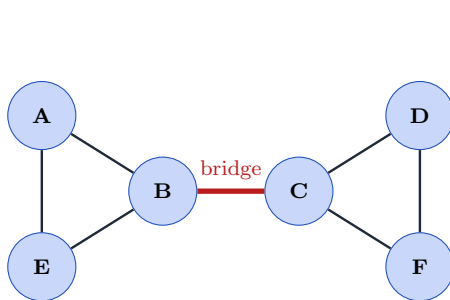
Definition: Critical node / critical link

A node (or link) whose failure alone **disconnects** the network, i.e. after removing it, some pair of remaining nodes can no longer reach each other. In graph theory: a critical node is an **articulation point (cut vertex)**, a critical link is a **bridge (cut edge)**.

What the slide says (Lecture 3, slide 6)

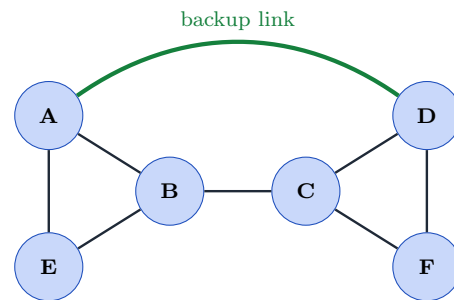
“Failure of a single entity (node or link) disconnects the entire network — nodes cannot reach each other.” The slide shows two graphs on nodes A–F and asks: *Is B a critical node? Is BC a critical link?*

Since the slide's exact edges are not in the text, here is an equivalent worked example that shows both cases.



Topology 1: two triangles joined by B–C

B and C are critical nodes; BC is a critical link



Topology 2: add A–D

No critical node or link (2-connected)

How to check (the exam method): remove the node/link and test whether every remaining pair is still connected.

- Topology 1, remove B: {A,E} are cut off from {C,D,F} $\Rightarrow$ B critical. Remove link BC: same split $\Rightarrow$ BC critical. Remove A: E still reaches B, so everything is connected $\Rightarrow$ A not critical.
- Topology 2, remove B: A–D still connects the two halves $\Rightarrow$ B not critical. Remove BC: A–D again saves it $\Rightarrow$ not critical.

Intuition: Why network designers care

Critical nodes/links are **single points of failure**. A fibre cut by construction work (the most common cause of outages in India) or a crashed router takes down everything that relied on it. The fix is **redundancy**: add links so there are at least two disjoint paths between every pair (2-connectivity). Designers weigh this against cost — recall the full mesh has *no* critical elements but $O(n^2)$ cost; a tree has minimum cost ($n - 1$ links) but *every* internal node and every link is critical. Real networks sit in between (rings, dual-homing, partial meshes).

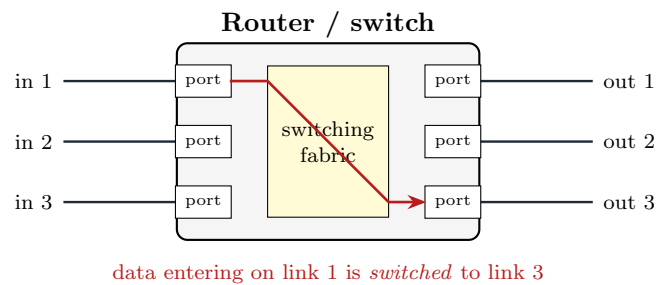
Beyond the slides: Useful facts

- In a **tree** with n nodes ($n - 1$ links), every link is a bridge and every non-leaf node is an articulation point.
- In a **ring** (cycle), no single node or link is critical; two failures can disconnect it.
- A link is a bridge *iff* it lies on no cycle.
- Articulation points and bridges can be found in $O(V + E)$ time with a DFS (Tarjan's algorithm) — a nice DSA link.

3.5 Routers and multi-link networks: switching

What the slide says (Lecture 3, slide 7)

- Routers need a way to move data across links.
- We use the term **switching** to denote physically moving data from one link to another.
- There are different possibilities to switch data between links...



3.6 Sharing a link: intuition with three bursty sources

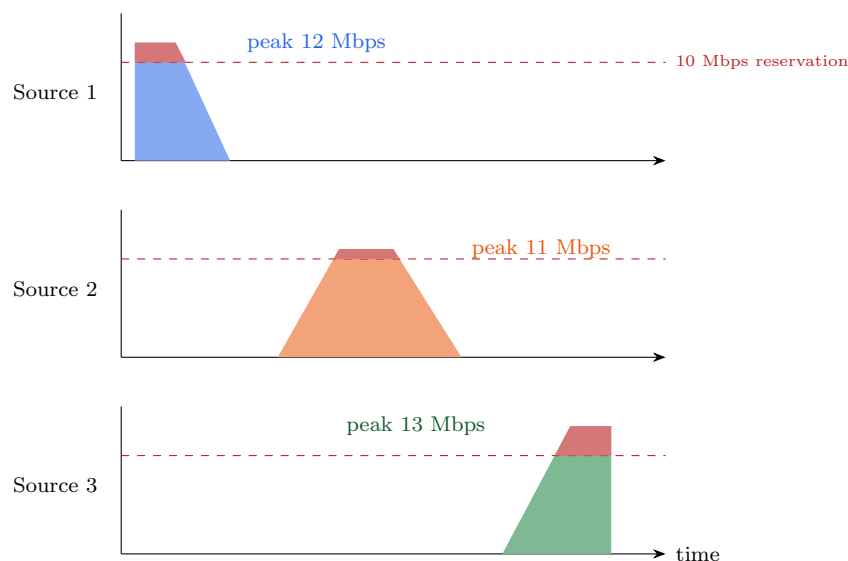
What the slide says (Lecture 3, slides 8–12)

Three sources with “bursty” traffic, peak rates 12, 11 and 13 Mbps, share a link of capacity 30 Mbps. Two options: **reservations** (each source gets a fixed 10 Mbps) vs **on demand**.

Definition: Bursty traffic

Traffic whose peak rate is much higher than its average rate: short bursts of activity separated by idle periods. Web browsing is the classic example (click, burst of data, then seconds of reading).

Reservations: red = traffic exceeding its 10 Mbps share



3.6.1 Option 1 — Reservations: “Frequent overloading”

Divide the 30 Mbps link into three fixed 10 Mbps slices, one per source.

Formula: Excess under reservation

For each source, excess = peak – reservation:

$$12 - 10 = 2 \text{ Mbps}, \quad 11 - 10 = 1 \text{ Mbps}, \quad 13 - 10 = 3 \text{ Mbps}.$$

Every source overloads *its own slice* during its burst — even though, at that moment, the other two slices are completely idle! Their 20 Mbps is wasted, while the bursting source must buffer or drop its excess.

3.6.2 Option 2 — On demand: “No overloading”

Let whoever is active use the whole 30 Mbps. Since the bursts in the slide happen at different times, each burst ($\text{peak} \leq 13 < 30$) fits comfortably.

Intuition: Why on-demand wins here

Reservations size capacity for the *peak* of each user, separately. On-demand sizes capacity for the *peak of the aggregate*. Because bursts of independent users rarely line up, the aggregate peak is much less than the sum of individual peaks: here the sum of peaks is $12 + 11 + 13 = 36 > 30$, yet the aggregate never exceeds 13.

3.6.3 Statistical multiplexing

Definition: Statistical multiplexing

Sharing a resource using the *statistics of demand*: capacity is allocated dynamically to whoever needs it, on the bet that not all users will be at their peak simultaneously.

What the slide says (Lecture 3, slide 11)

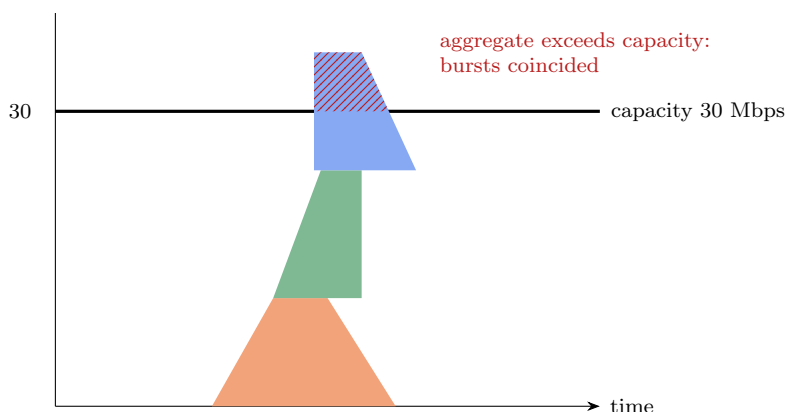
- Switching on-demand exploits statistical multiplexing better than reservations.
- Sharing using the statistics of demand.
- Good for bursty traffic (average $\ll$ peak demand).
- Similar to insurance, with the same failure mode.

Intuition: The insurance analogy — and its failure mode

An insurance company collects small premiums from millions of people and can pay large claims because only a few claim at a time. It does *not* hold enough money to pay everyone at once. The failure mode: a **correlated event** (flood, pandemic) where everyone claims simultaneously — the company goes bust.

Statistical multiplexing is identical: it works because users are independent and bursty. It fails under **correlated demand**: IPL final on Hotstar, a Flipkart Big Billion Days sale, exam results day on a university server, everyone joining a 9 AM Zoom class. Then the aggregate exceeds capacity.

3.6.4 What do we do under overload?



When bursts coincide (here roughly $11 + 13 + 12 = 36 > 30$ Mbps), the router has three options:

1. **Buffer** the excess in a queue and send it when the burst ends $\Rightarrow$ extra (queuing) delay.
2. **Drop** packets when the buffer is full $\Rightarrow$ loss (best effort!).
3. **Signal** senders to slow down $\Rightarrow$ congestion control (Challenge #4).

This connects everything: statistical multiplexing is *why* the Internet has queues, queuing delay, packet loss, and the need for congestion control and packet scheduling. It was a deliberate trade: much better average efficiency in exchange for occasional overload.

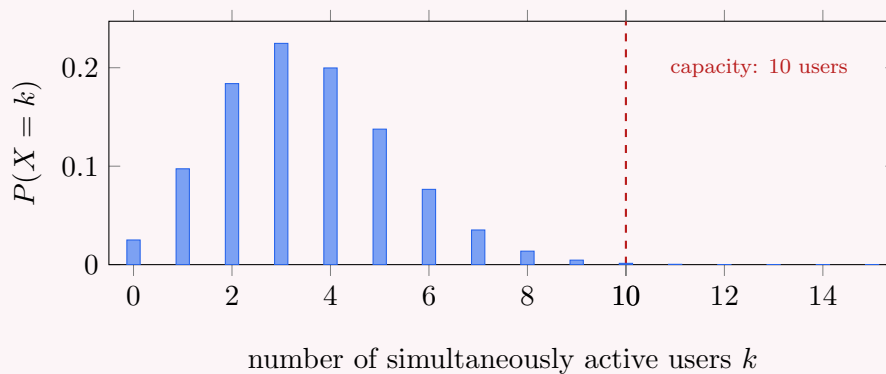
Beyond the slides: Quantifying statistical multiplexing (KR Ch. 1 classic example)

A 1 Mbps link is shared by users who each send at 100 kbps when active, and are active 10% of the time (independently).

- **Reservation (circuit switching):** each user needs 100 kbps reserved $\Rightarrow$ at most $\frac{1 \text{ Mbps}}{100 \text{ kbps}} = 10$ users.
- **On demand (packet switching):** with $N = 35$ users, overload happens only if more than 10 are active at once. The number active $X \sim \text{Binomial}(N=35, p=0.1)$:

$$P(X > 10) = \sum_{k=11}^{35} \binom{35}{k} (0.1)^k (0.9)^{35-k} \approx 0.0004.$$

So $3.5\times$ as many users are supported, with overload only 0.04% of the time.

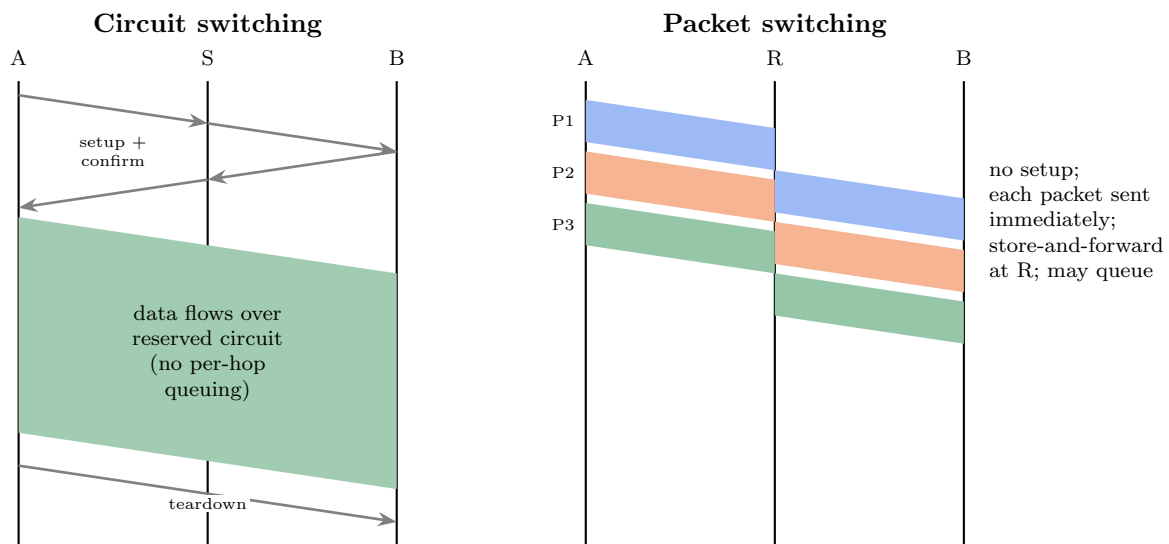


The mass to the right of the dashed line is tiny. That is statistical multiplexing in one picture.

3.7 Two approaches to sharing: circuit switching vs packet switching

What the slide says (Lecture 3, final slide)

- Reservations $\rightarrow$ **circuit switching**.
- On demand $\rightarrow$ **packet switching**.
- How are these implemented? (next lectures)



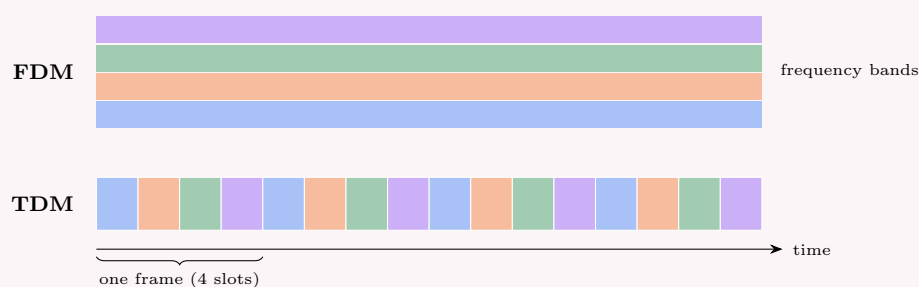
	Circuit switching (reservations)	Packet switching (on demand)
Resource allocation	Dedicated, fixed share reserved end-to-end for the whole call	Shared, used as needed packet by packet
Setup	Required before data (call setup)	None — just send
Guarantees	Guaranteed rate, constant delay	None: variable delay, possible loss
Efficiency for bursty traffic	Poor — idle reserved capacity wasted	High — statistical multiplexing
State in the network	Per-call state in every switch	None per flow (only forwarding tables)
Failure behaviour	Switch fails $\Rightarrow$ call dropped	Packets reroute around failures
Overload behaviour	New calls <i>blocked</i> (“all lines busy”)	Everyone slows down: queues, delay, loss
Examples	Traditional telephone network (PSTN), old GSM voice	The Internet (IP), Ethernet

Intuition: Which one when?

Circuit switching is like booking an entire train compartment: guaranteed seats, but empty seats are wasted if your friends don't show up. Packet switching is like the unreserved general compartment: very efficient use of seats, but on festival days it's overcrowded and you may not get on. For the *voice calls* of the 20th century (constant rate, long duration, very delay-sensitive), circuits were natural. For *computer data* (bursty, average $\ll$ peak), packet switching wins decisively — which is why the Internet is packet-switched. Implementations of both (TDM/FDM circuits; store-and-forward packets) come next week.

Beyond the slides: Preview: how circuits divide a link

FDM (frequency-division multiplexing): each circuit gets a frequency band (like FM stations). **TDM (time-division multiplexing)**: time is cut into frames of N slots; each circuit gets one slot per frame, so each gets rate R/N regardless of whether it's used.



Formula: Per-circuit rate under TDM/FDM with N circuits

$$R_{\text{circuit}} = \frac{R}{N} \quad [\text{bits/s}]$$

$$\text{File transfer time over a circuit} = t_{\text{setup}} + \frac{L}{R/N}.$$

Summary: Module 3 in five lines

1. Bits become physical signals by modulating a carrier's amplitude, frequency or phase; decoding reverses it.
2. Full mesh needs $n(n-1)/2$ links; a shared link gives each node R/n ; switched networks scale.
3. Critical nodes/links (cut vertices/bridges) are single points of failure; redundancy removes them.
4. Bursty traffic is served far better on demand (statistical multiplexing) than by fixed reservations — until correlated bursts cause overload (buffer, drop, or slow senders).
5. Reservations $\leftrightarrow$ circuit switching; on demand $\leftrightarrow$ packet switching.

4 Formula Sheet

Quantity	Formula	Units
Propagation delay	$d_{\text{prop}} = D/s$ ($s \approx 3 \times 10^8$ vacuum, $\approx 2 \times 10^8$ fibre/copper)	s = m / (m/s)
Transmission delay	$d_{\text{trans}} = L/R$	s = bits / (bits/s)
Nodal delay	$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$	s
End-to-end (store-and-forward, N equal links, no queuing)	$d = N \frac{L}{R} + \sum_i \frac{D_i}{s} + (N-1)d_{\text{proc}}$	s
RTT (ideal)	RTT = $2 d_{\text{prop}}$ (+ delays both ways)	s
Throughput	data/time ; path throughput = $\min(R_1, \dots, R_N)$	bits/s
Bandwidth-delay product	BDP = $R \times \text{RTT}$	bits
Full-mesh links	$n(n-1)/2$	links
Shared-link per-node capacity	R/n	bits/s
TDM/FDM circuit rate	R/N ; transfer time = $t_{\text{setup}} + L/(R/N)$	bits/s ; s
Bits per symbol	$\log_2 M$; $R_{\text{bit}} = R_{\text{sym}} \log_2 M$	bits/s
Stat. mux overload	$P(X > C) = \sum_{k=C+1}^N \binom{N}{k} p^k (1-p)^{N-k}$	—
Unit conversions	1 B = 8 b; 1 Mbps = 10^6 bit/s; 1 ms = 10^{-3} s	

Common pitfall: KB: 1000 or 1024?

In networking, rates use SI prefixes (1 Mbps = 10^6 bit/s exactly). File sizes are ambiguous (KB = 1000 or KiB = 1024). Unless told otherwise, use 1 KB = 1000 bytes in this course's numericals, and state your assumption.

5 Numerical Problems with Worked Solutions**Problem 1.** *Propagation delay and the ping inflation*

IIT Madras and IIT Delhi are ≈ 2000 km apart (straight line). (a) Compute the one-way propagation delay in vacuum and in optical fibre ($s = 2 \times 10^8$ m/s). (b) Compute the corresponding ideal RTTs. (c) If **ping** reports 160 ms, what is the inflation factor relative to the slide's 6.67 ms, and relative to the fibre RTT?

Solution

(a) $D = 2000 \text{ km} = 2 \times 10^6 \text{ m}$.

$$d_{\text{vac}} = \frac{2 \times 10^6 \text{ m}}{3 \times 10^8 \text{ m/s}} = 6.67 \text{ ms}, \quad d_{\text{fibre}} = \frac{2 \times 10^6 \text{ m}}{2 \times 10^8 \text{ m/s}} = 10 \text{ ms}.$$

(b) $\text{RTT} = 2d_{\text{prop}}$: vacuum RTT = 13.3 ms, fibre RTT = 20 ms.

(c) Relative to 6.67 ms: $160/6.67 = 24\times$ (inside the slide's 20–30 $\times$ range). Relative to fibre RTT: $160/20 = 8\times$. So 3 $\times$ of the “24 $\times$ ” is explained by RTT-vs-one-way (2 $\times$) and fibre speed (1.5 $\times$); the remaining 8 $\times$ is path stretch, queuing, processing and host delays.

Problem 2. *Transmission delay*

Find the transmission delay of a 1500-byte packet on links of (a) 100 kbps, (b) 10 Mbps, (c) 1 Gbps.

Solution

$$L = 1500 \times 8 = 12,000 \text{ bits}. \quad d_{\text{trans}} = L/R \quad [s = b/(b/s)].$$

(a) $12000/10^5 = 0.12 \text{ s} = 120 \text{ ms}$.

(b) $12000/10^7 = 1.2 \text{ ms}$.

(c) $12000/10^9 = 12 \mu\text{s}$.

Observation: on fast links transmission delay becomes negligible and propagation dominates; on slow links the reverse.

Problem 3. *End-to-end delay over the X–A–B–Y path*

In the path X–A–B–Y (two routers), the three links have lengths 1 km, 2000 km and 1 km, all with rate 10 Mbps and $s = 2 \times 10^8$ m/s. A 1500-byte packet is sent from X to Y. Each router takes 0.1 ms to process a packet; there is no queuing. Routers are store-and-forward. Find the total delay.

Solution

Store-and-forward means a router must receive the *entire* packet before forwarding it, so the packet is transmitted 3 times (by X, A and B).

- Transmission: $3 \times \frac{12000}{10^7} = 3 \times 1.2 = 3.6 \text{ ms}$.
- Propagation: $\frac{(1+2000+1) \times 10^3}{2 \times 10^8} = \frac{2.002 \times 10^6}{2 \times 10^8} = 10.01 \text{ ms}$.
- Processing: $2 \times 0.1 = 0.2 \text{ ms}$.

Total = $3.6 + 10.01 + 0.2 = \mathbf{13.81 \text{ ms}}$.

Problem 4. *When does transmission equal propagation?*

For a 1500-byte packet on a 100 Mbps link with $s = 2 \times 10^8$ m/s, find the link length D at which $d_{\text{trans}} = d_{\text{prop}}$. Interpret.

Solution

$d_{\text{trans}} = 12000/10^8 = 0.12$ ms. Setting $D/s = L/R$:

$$D = s \frac{L}{R} = 2 \times 10^8 \times 1.2 \times 10^{-4} = 2.4 \times 10^4 \text{ m} = 24 \text{ km}.$$

Interpretation: for links shorter than 24 km (LANs, campus), transmission dominates; for longer links (Madras–Delhi), propagation dominates. Another view: sL/R is the physical “length” of the packet on the wire — at 100 Mbps, one 1500-byte packet is 24 km long!

Problem 5. *Throughput from the wget experiment*

(a) Convert the slide’s 200 KB in 754 ms to Mbps. (b) If the path had 100 Mbps capacity and zero latency, how long would the download take? (c) What does this say about the bottleneck?

Solution

(a) $\frac{200 \times 10^3 \times 8 \text{ bits}}{0.754 \text{ s}} = \frac{1.6 \times 10^6}{0.754} = 2.12 \times 10^6 \text{ bit/s} = 2.12 \text{ Mbps}$ (i.e. 265 KB/s).

(b) $\frac{1.6 \times 10^6}{10^8} = 16$ ms.

(c) 754 ms vs 16 ms: $\approx 98\%$ of the time is *not* spent pushing bits; it’s spent waiting on round trips (handshake, request, TCP slow start). For short transfers, latency, not bandwidth, is the bottleneck.

Problem 6. *Round trips under slow start*

A client downloads a 500 KB file. RTT = 40 ms, TCP initial window = 14.6 KB, window doubles each RTT, one RTT for connection setup before the request. Ignoring transmission time, estimate the download time.

Solution

Cumulative data after each data-RTT: 14.6, 43.8, 102.2, 219.0, 452.6, 919.8 KB. The 500 KB mark is passed in the **6th** data round. Total RTTs = 1 (setup) + 6 = 7. Time $\approx 7 \times 40 = \mathbf{280}$ ms. Average throughput = $\frac{500 \times 8 \text{ kb}}{0.28 \text{ s}} \approx 14.3 \text{ Mbps}$ — notice it depends on RTT, not link speed.

Problem 7. *Distance bound from ping*

The minimum RTT to a server, over many pings, is 12 ms. Assuming fibre (2×10^8 m/s), what is the maximum possible distance to the server? Why use the minimum RTT?

Solution

Even with zero queuing/processing, $\text{RTT} \geq 2D/s$, so $D \leq s \cdot \text{RTT}/2 = 2 \times 10^8 \times 0.006 = 1.2 \times 10^6 \text{ m} = \mathbf{1200}$ km. We use the minimum RTT because it contains the least queuing delay, so it’s closest to pure propagation. (This is how “geolocation by latency” works; it proves, e.g., that a server claimed to be in the US cannot be, if RTT from Chennai is 12 ms.)

Problem 8. *Bottleneck throughput*

A file of 50 MB goes over three links in series with rates 100 Mbps, 10 Mbps and 1 Gbps. Estimate the transfer time (ignore latency).

Solution

End-to-end throughput is limited by the slowest link: $\min(100, 10, 1000) = 10$ Mbps. Time $= \frac{50 \times 10^6 \times 8}{10^7} = 40$ s. Upgrading the 1 Gbps link would do nothing; upgrading the 10 Mbps link would help until the 100 Mbps link becomes the bottleneck.

Problem 9. *Bandwidth-delay product*

A 1 Gbps path has RTT 20 ms. (a) Compute the BDP in bits and bytes. (b) What does it mean for a sender?

Solution

- (a) $\text{BDP} = R \times \text{RTT} = 10^9 \times 0.02 = 2 \times 10^7$ bits = 2.5 MB [bits = bits/s · s].
 (b) It's the amount of data "in flight" in the pipe during one round trip. To keep the link fully busy, the sender must have 2.5 MB sent-but-not-yet-acknowledged. With a smaller window (e.g. 64 KB), throughput is capped at $\frac{64 \times 10^3 \times 8}{0.02} = 25.6$ Mbps — only 2.6% of the link.

Problem 10. *Full mesh vs switched*

(a) How many links does a full mesh of 20 hosts need? (b) How many new links when a 21st host joins? (c) With 50 hosts? (d) A star (all hosts to one switch) with 50 hosts?

Solution

- (a) $\frac{20 \times 19}{2} = 190$. (b) The new host connects to all 20 existing ones: 20 new links (check: $\frac{21 \times 20}{2} = 210 = 190 + 20$). (c) $\frac{50 \times 49}{2} = 1225$. (d) 50 links — but the switch is now a *critical node*.

Problem 11. *Shared link*

A 100 Mbps shared link serves 25 active hosts. (a) Best-case average per-host rate? (b) Maximum number of hosts if each needs at least 2 Mbps?

Solution

- (a) $100/25 = 4$ Mbps (in practice less, due to collisions/MAC overhead). (b) $100/n \geq 2 \Rightarrow n \leq 50$.

Problem 12. *The three bursty sources*

Three sources with peaks 12, 11 and 13 Mbps share a 30 Mbps link. (a) Under a reservation of 10 Mbps each, what is each source's excess at its peak, and how much capacity is idle during source 1's burst (others silent)? (b) Under on-demand sharing, suppose all three bursts overlap for 0.5 s. What buffer size is needed to avoid loss?

Solution

- (a) Excess: 2, 1, 3 Mbps. During source 1's burst, sources 2 and 3 are silent but their reserved 20 Mbps is *idle* — wasted while source 1 overflows.
 (b) Aggregate = 12 + 11 + 13 = 36 Mbps; excess = 6 Mbps. Over 0.5 s the excess volume is $6 \times 10^6 \times 0.5 = 3 \times 10^6$ bits = 375 KB [bits = bits/s · s]. The router needs ≥ 375 KB of buffer (or it will drop). The last buffered bit waits $3 \times 10^6 / 30 \times 10^6 = 0.1$ s extra after the burst — that's queuing delay.

Problem 13. *Statistical multiplexing gain*

A 1 Mbps link; each user sends at 100 kbps when active and is active 10% of the time, independently. (a) Users supported with circuit switching? (b) With packet switching and 35 users, probability of overload? (c) With 50 users?

Solution

(a) $1000/100 = 10$ users.

(b) Overload iff $X > 10$ active, $X \sim \text{Bin}(35, 0.1)$. Mean = 3.5. $P(X > 10) = \sum_{k=11}^{35} \binom{35}{k} 0.1^k 0.9^{35-k} \approx 4.2 \times 10^{-4}$.

(c) $X \sim \text{Bin}(50, 0.1)$, mean 5: $P(X > 10) \approx 9.4 \times 10^{-3}$ (about 1%).

Interpretation: packet switching supports $3.5\times$ more users with 0.04% overload probability, or $5\times$ more with 1%. The gain shrinks if users' activity becomes correlated (the insurance failure mode).

Problem 14. Circuit-switched file transfer (TDM)

A 640,000-bit file is sent over a circuit-switched network. The link uses TDM with 24 slots and total rate 1.536 Mbps. Circuit setup takes 500 ms. How long does the transfer take? Compare with an idle packet-switched link using the full rate.

Solution

Per-circuit rate = $1.536 \times 10^6 / 24 = 64$ kbps. Transfer = $0.5 + \frac{640000}{64000} = 0.5 + 10 = 10.5$ s. Packet switching with the full 1.536 Mbps (if other users are idle): $\frac{640000}{1.536 \times 10^6} = 0.417$ s — $25\times$ faster. The circuit's fixed $1/24$ share is wasted when the other 23 slots are idle.

Problem 15. Modulation and bit rate

A modem operates at 2400 symbols/s. (a) Bit rate with binary PSK? (b) With 16 distinguishable signal states? (c) How many states are needed for 19,200 bit/s?

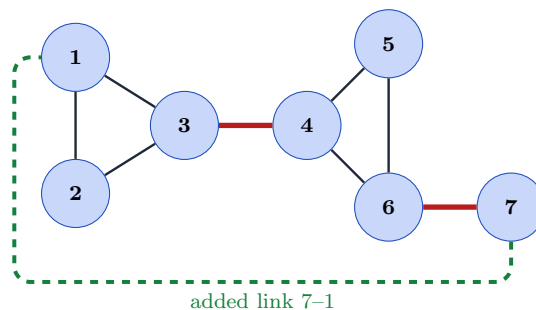
Solution

(a) $2400 \times \log_2 2 = 2400$ bit/s. (b) $2400 \times \log_2 16 = 9600$ bit/s. (c) $19200/2400 = 8$ bits/symbol $\Rightarrow M = 2^8 = 256$ states.

Problem 16. Critical nodes and links

A network has nodes 1–7 and links 1–2, 2–3, 3–1, 3–4, 4–5, 5–6, 6–4, 6–7. (a) Find all critical nodes and critical links. (b) Add the fewest links to eliminate all of them.

Solution



(a) **Critical links (bridges):** 3–4 and 6–7 (neither lies on any cycle). **Critical nodes:** 3 (removing it isolates $\{1,2\}$), 4 (isolates $\{5,6,7\}$ from $\{1,2,3\}$), 6 (isolates 7). Nodes 1, 2, 5, 7 are not critical.

(b) One link suffices: add 7–1. It creates the cycle 1–3–4–6–7–1, which contains both bridges,

and removing any single node of $\{3, 4, 6\}$ still leaves a path around the other side of this cycle.

6 Subjective Questions with Model Answers

S1. Distinguish the two meanings of “the Internet”. Which one does this course study, and why is the distinction useful?

(1) The ecosystem of applications and companies (Google, Netflix, UPI, trading). (2) The infrastructure — links, routers and protocols such as TCP, IP, BGP, DNS, OSPF, DHCP, ARP — that connects computing devices. The course studies (2). The distinction matters because the infrastructure is *general-purpose*: it must serve all applications, including ones not yet invented, so its design principles (best effort, layering, narrow waist) are about generality rather than any one app.

S2. List and explain the three problems that must be solved even on a single shared link.

(i) **Encoding/decoding**: mapping bits to physical signals (modulating amplitude, frequency or phase of a carrier) and recovering them. (ii) **Medium access control**: deciding who transmits when, because simultaneous transmissions on a shared medium collide and become undecodable. (iii) **Error detection and correction**: noise corrupts bits; redundancy (checksums, CRC, error-correcting codes) lets the receiver detect or repair errors. Also needed: link-level (MAC) addresses so receivers know which frames are theirs.

S3. Why can’t the whole world be connected by one shared link? Why can’t it be a full mesh?

Shared link: signals attenuate with distance, so a single link has limited physical reach; its capacity R is divided among n users (R/n each), and collisions/MAC overhead grow with n . Full mesh: requires $n(n-1)/2 = O(n^2)$ links and $n-1$ ports per host; for billions of hosts this is physically and economically impossible, and adding a host requires touching every existing host. Switched networks (hosts connect to switches/routers, which are interconnected) scale with roughly linear cost.

S4. Explain best-effort delivery. What are its advantages, and who compensates for its weaknesses?

Best effort: the network tries to deliver packets but guarantees nothing — packets may be lost, corrupted, reordered (or duplicated), with variable delay. Advantages: the core is simple to build (no reliability, ordering or performance mechanisms, no per-connection state), almost any medium can carry individual packets (so heterogeneous networks interconnect easily), failures are survived by rerouting, and new applications can be deployed without changing the network. The weaknesses are compensated at the endpoints by transport software (e.g. TCP) using sequence numbers, ACKs, timeouts/retransmissions and checksums — the end-to-end principle.

S5. Differentiate congestion control from packet scheduling.

Both deal with overload, from different vantage points. **Congestion control** runs at the *source endpoints* (in the transport stack): the sender infers congestion from network signals (loss, delay, ECN marks) and adjusts its sending rate. **Packet scheduling** runs *inside routers*: given packets queued for a constrained output link, it decides which to send next (FIFO, priority, fair queuing). Its companion, **buffer management**, decides which packets to drop when the buffer is full. Congestion control reduces offered load; scheduling allocates whatever capacity exists.

S6. The slide says ping from IIT Madras to IIT Delhi shows $20\text{--}30\times$ inflation over D/c . Explain all contributing factors.

(1) Ping measures RTT, but 6.6 ms is one-way: $\times 2$. (2) Light in fibre travels at $\approx c/1.5$: $\times 1.5$. (3) Path stretch: cables follow roads/railways and traffic may traverse distant exchange points (possibly abroad) rather than the straight line. (4) Per-hop processing delays at each router. (5) Queuing delays in router buffers (variable; causes jitter). (6) Transmission delays at each store-and-forward hop. (7) End-host delays in generating the ICMP reply. Items (1)–(2) alone explain $3\times$; the rest comes from the network’s structure — the “bottleneck is the infrastructure”.

S7. Why is a 200 KB download’s throughput (265 KB/s) far below the link capacity? What general lesson follows?

A short transfer is dominated by round trips: DNS lookup, TCP handshake (1 RTT), HTTP request/response (1 RTT), and TCP slow start, where the window starts small and doubles per RTT so the transfer ends before full speed is reached. With $\text{RTT} \approx 150$ ms, about 5 RTTs explain the 754 ms. Lesson: latency and throughput are distinct; for small objects (most web traffic) reducing RTT (CDNs, caching, connection reuse) matters more than adding bandwidth.

S8. What is the “narrow waist” of the Internet hourglass? Discuss its benefits and costs.

The single network-layer protocol, IP, sits between many applications/transport protocols above and many link technologies below. Benefits: interoperability (any app over any link), $M + N$ instead of $M \times N$ adaptations, independent innovation above and below (new apps and new link technologies deploy without coordination). Costs: the waist is extremely hard to change (IPv6 adoption has taken decades) and its limitations — notably best-effort service with no delay guarantees — are inherited by every application.

S9. Explain amplitude, frequency and phase modulation with a sketch, and compare their noise robustness.

A carrier $A \sin(2\pi ft + \phi)$ has three parameters. ASK varies A (e.g. carrier on for 1, off for 0); FSK uses two frequencies f_1, f_0 ; PSK shifts phase (0 for 0, π for 1). (Sketch as in Module 3.) ASK is most susceptible to noise because additive noise directly perturbs amplitude; FSK is more robust since noise rarely changes frequency; binary PSK separates the two symbols maximally (s vs $-s$) and is the most robust for a given power. Practical systems combine amplitude and phase (QAM) to carry $\log_2 M$ bits per symbol.

S10. What are critical nodes and links? How are they found, and how can a network designer eliminate them?

A critical node (articulation point) or critical link (bridge) is one whose single failure disconnects the network. To check, remove it and test whether all remaining pairs are still connected (systematically: DFS-based Tarjan algorithm in $O(V + E)$). A link is a bridge iff it lies on no cycle. Designers eliminate them by adding redundant links so every node/link lies on a cycle (2-connectivity): rings, dual-homed hosts, partial meshes — trading extra cost for fault tolerance.

S11. Explain statistical multiplexing using the three-source example, and describe its failure mode.

Sources with peaks 12, 11, 13 Mbps share 30 Mbps. With fixed reservations of 10 Mbps each, every source overflows its slice during its burst while the other slices sit idle — frequent overloading and

waste. On demand, whichever source is active uses the full 30 Mbps; since bursts rarely coincide, no overload occurs. This is statistical multiplexing: capacity is provisioned for the aggregate's statistics rather than the sum of peaks, ideal when average $\ll$ peak. Like insurance, it fails under correlated demand (all bursts at once, e.g. 36 Mbps $>$ 30 Mbps): the router must buffer (delay), drop (loss) or make senders slow down (congestion control).

S12. Compare circuit switching and packet switching. Why is the Internet packet-switched?

Circuit switching = reservations: a call-setup phase reserves a fixed share (via FDM/TDM) on every link end-to-end; the flow gets guaranteed rate and constant delay; idle reserved capacity is wasted; overload means new calls are blocked; switches keep per-call state. Packet switching = on demand: no setup, each packet uses full link rate when available, store-and-forward with queues; no guarantees (variable delay, loss); excellent efficiency for bursty traffic; no per-flow state; failures handled by rerouting. Computer traffic is bursty (average $\ll$ peak), so packet switching's statistical multiplexing supports many more users on the same links (e.g. 35 vs 10 in the classic example), making it the right choice for the Internet.

S13. Map each of the five challenges from Lecture 1 to where in the system it is solved.

#1 Multiple hosts on one link (encoding, MAC, errors) $\rightarrow$ link/physical layer, in every NIC. #2 Hosts across multiple links (routing) $\rightarrow$ network layer, routers and routing protocols. #3 Application guarantees (reliable, ordered delivery) $\rightarrow$ transport layer at endpoints. #4 Sending rate (congestion control) $\rightarrow$ transport layer at source endpoints. #5 Resource crunch (packet scheduling, buffer management) $\rightarrow$ inside routers, at output queues.

7 Multiple-Choice Questions with Explanations

Q1. In this course, “the Internet” primarily refers to:

- (a) The set of applications like Google and Netflix
- (b) The infrastructure (links, routers, protocols like TCP/IP/BGP/DNS) connecting devices
- (c) The World Wide Web
- (d) Cloud providers like AWS

(b). Lecture 1 explicitly adopts the second definition — the infrastructure. The Web is just one application.

Q2. Which of the following is *not* a host/endpoint?

- (a) A laptop running a browser
- (b) A smartphone
- (c) A WiFi router forwarding packets
- (d) A server hosting a website

(c). The WiFi router is a device “in the middle of the network” that forwards data; hosts run user applications.

Q3. A MAC address is best described as:

- (a) A link-level address identifying an endpoint on a link
- (b) A globally routable Internet address
- (c) An address used by Apple Macintosh computers only
- (d) The address of a DNS server

(a). MAC = Medium Access Control; it's the link-level address used to pick out the receiver on a shared link.

Q4. Which is *not* one of the single-link problems listed in Lecture 1?

- (a) Encoding/decoding digital data to physical signals
- (b) Deciding who speaks (medium access control)
- (c) Detecting and correcting errors
- (d) Choosing a path through multiple routers

(d). Path selection is the routing problem, which arises only with multiple links (Challenge #2).

Q5. Why can't every computer in the world be on the same link?

- (a) MAC addresses would run out
- (b) Physical limits on power/distance and capacity sharing
- (c) Routers would be too slow
- (d) TCP doesn't allow it

(b). Signals attenuate with distance and a shared link's capacity is divided among all users.

Q6. "Best-effort delivery" means the network:

- (a) Guarantees delivery but not ordering
- (b) Guarantees ordering but not delivery
- (c) Makes no guarantees about delivery, integrity, ordering or timing
- (d) Retransmits lost packets automatically

(c). Packets may be lost, corrupted or reordered; retransmission is done by endpoints, not the network.

Q7. A key advantage of best-effort networks is:

- (a) Guaranteed low latency
- (b) No need for transport protocols
- (c) Packets are never lost
- (d) The network is simple to build and almost any medium can deliver packets

(d). Stated directly on the slide; simplicity is why the early Internet thrived.

Q8. Reliable data delivery on the Internet is implemented by:

- (a) Routers in the core
- (b) Transport software on the endpoints
- (c) The physical layer
- (d) DNS

(b). "Transport software on the endpoint oversees implementing guarantees on top of an unreliable network."

Q9. The problem of how fast endpoints should send data is called:

- (a) Flow control
- (b) Routing
- (c) Congestion control
- (d) Packet scheduling

(c). Congestion control (Challenge #4). Flow control protects the receiver, not the network.

Q10. Deciding which queued packet a router transmits next over a constrained link is:

- (a) Congestion control
- (b) Packet scheduling
- (c) Medium access control
- (d) Encoding

(b). Challenge #5; the related problem of what to drop is buffer management.

Q11. The one-way speed-of-light delay over 2000 km is approximately:

- (a) 0.66 ms
- (b) 6.6 ms
- (c) 66 ms
- (d) 660 ms

(b). $2 \times 10^6 / 3 \times 10^8 = 6.67$ ms.

Q12. ping reports:

- (a) One-way delay
- (b) Throughput
- (c) Transmission delay only
- (d) Round-trip time

(d). Ping sends an ICMP echo request and times the reply — a round trip. Comparing it with one-way D/c already introduces a factor of 2.

Q13. Light in optical fibre travels at roughly:

- (a) 2×10^8 m/s
- (b) 3×10^8 m/s
- (c) 3×10^6 m/s
- (d) 1×10^{10} m/s

(a). Refractive index ≈ 1.5 gives $c/1.5 \approx 2 \times 10^8$ m/s.

Q14. 200 KB downloaded in 754 ms corresponds to an average throughput of about:

- (a) 265 kbps
- (b) 2.12 Mbps
- (c) 21.2 Mbps
- (d) 0.265 Mbps

(b). $200 \text{ KB} / 0.754 \text{ s} = 265 \text{ KB/s} = 265 \times 8 = 2120 \text{ kbps}$. Trap: (a) forgets the bytes→bits factor.

Q15. The transmission delay of a packet depends on:

- (a) Distance between routers
- (b) Packet length and link rate
- (c) Propagation speed
- (d) Number of routers

(b). $d_{\text{trans}} = L/R$; distance affects only propagation delay.

Q16. Which delay component is the most variable and depends on traffic load?

- (a) Propagation

- (b) Transmission
- (c) Queuing
- (d) Processing

(c). Queuing delay varies with how many packets are ahead in the buffer; it causes jitter.

Q17. In the Internet hourglass, the “narrow waist” is:

- (a) TCP
- (b) HTTP
- (c) IP
- (d) Ethernet

(c). All applications and all link technologies meet at a single network-layer protocol, IP.

Q18. In the protocol graph shown in Lecture 2, RTP runs over:

- (a) TCP
- (b) UDP
- (c) Ethernet directly
- (d) HTTP

(b). Real-time media prefers timeliness over reliability, so RTP uses UDP.

Q19. Converting bits into physical signals by varying carrier properties is called:

- (a) Multiplexing
- (b) Modulation
- (c) Routing
- (d) Switching

(b). Modulation = encoding; demodulation = decoding.

Q20. In phase modulation (binary PSK), a bit change is represented by:

- (a) A change in amplitude
- (b) A change in frequency
- (c) A shift in the phase of the carrier (e.g. by π)
- (d) Turning the carrier off

(c). The figure labels “phase changes” at bit boundaries where the value changes.

Q21. A full mesh of 10 nodes requires how many links?

- (a) 10
- (b) 20
- (c) 45
- (d) 90

(c). $10 \times 9/2 = 45$. (d) double-counts each link.

Q22. On a single shared link with n active nodes, each node’s capacity scales as:

- (a) $O(1)$
- (b) $O(1/n)$
- (c) $O(n)$
- (d) $O(n^2)$

(b). Capacity R divided among n nodes.

Q23. Which topology enables efficient scaling to many end-systems?

- (a) Full mesh
- (b) Single shared bus
- (c) Switched network
- (d) Point-to-point pair

(c). Slide: “Switched networks enable efficient scaling!”

Q24. A critical link is one whose failure:

- (a) Disconnects the network
- (b) Increases latency
- (c) Causes packet reordering
- (d) Reduces throughput by half

(a). By definition (a bridge / cut edge).

Q25. In a tree network with n nodes:

- (a) No link is critical
- (b) Exactly one link is critical
- (c) Every link is critical
- (d) Only leaf links are critical

(c). A tree has no cycles, so every edge is a bridge.

Q26. “Switching” refers to:

- (a) Changing ISPs
- (b) Physically moving data from one link to another
- (c) Encoding bits into signals
- (d) Switching between TCP and UDP

(b). Definition from Lecture 3, slide 7.

Q27. In the three-source example with 10 Mbps reservations each, the result is:

- (a) No overloading
- (b) Frequent overloading while other reservations sit idle
- (c) Link capacity is exceeded at all times
- (d) Only source 3 overloads

(b). Every source’s peak (12, 11, 13) exceeds 10; the others’ idle share is wasted.

Q28. Statistical multiplexing works best when traffic is:

- (a) Constant bit rate
- (b) Perfectly correlated across users
- (c) Bursty, with average much less than peak
- (d) Always at peak

(c). Slide: “Good for bursty traffic (average $\ll$ peak demand).” Correlation is its failure mode.

Q29. The “insurance” analogy for statistical multiplexing highlights that it fails when:

- (a) Users are independent
- (b) Many users demand resources simultaneously (correlated demand)
- (c) Packets are small
- (d) Links are wired

(b). Like everyone claiming insurance after a flood; e.g. everyone streaming a cricket final.

Q30. Reservations and on-demand sharing correspond, respectively, to:

- (a) Packet switching and circuit switching
- (b) TCP and UDP
- (c) FDM and TDM
- (d) Circuit switching and packet switching

(d). Stated on the final slide of Lecture 3.

Q31. Under overload, a packet-switched router can do all of the following *except*:

- (a) Buffer excess packets
- (b) Drop packets when the buffer is full
- (c) Guarantee every flow its peak rate
- (d) Mark/signal senders to slow down

(c). Guaranteeing peak rates is what reservations try to do; packet switching offers no such guarantee.

Q32. With users active 10% of the time at 100 kbps on a 1 Mbps link, circuit switching supports:

- (a) 10 users
- (b) 35 users
- (c) 100 users
- (d) 1 user

(a). $1 \text{ Mbps} / 100 \text{ kbps} = 10$ reserved circuits; packet switching can support ≈ 35 with tiny overload probability.

Q33. A 1500-byte packet on a 100 Mbps link has transmission delay:

- (a) $1.2 \mu\text{s}$
- (b) $12 \mu\text{s}$
- (c) $120 \mu\text{s}$
- (d) 1.2 ms

(c). $12000 / 10^8 = 1.2 \times 10^{-4} \text{ s} = 120 \mu\text{s}$.

Q34. Which programming language is used for the socket-programming project (P2)?

- (a) Python
- (b) Java
- (c) C
- (d) Go

(c). P1 and P3 use Python; P2 (sockets) uses C, following Stevens' *UNIX Network Programming*.

Summary: Final checklist — can you...

- define host, router, link, packet, network, address, MAC address, best effort?
- state the five challenges and where each is solved?

- compute d_{prop} , d_{trans} , RTT, throughput, BDP with correct units?
- explain every factor in the $20\text{--}30\times$ ping inflation and the 754 ms wget result?
- draw the hourglass and explain the narrow waist?
- sketch ASK/FSK/PSK for a given bit string?
- count mesh links, per-node shared capacity, and find critical nodes/links?
- explain statistical multiplexing, its failure mode, and circuit vs packet switching?